



Slow feature subspace: A video representation based on slow feature analysis for action recognition

Suzana Rita Alves Belezã^{a,*}, Erica K. Shimomoto^b, Lincon S. Souza^b, Kazuhiro Fukui^a

^a University of Tsukuba, Department of Computer Science, 305-8573, Tsukuba, Ibaraki, Japan

^b National Institute of Advanced Industrial Science and Technology (AIST), 135-0064, Koto-ku, Tokyo, Japan

ARTICLE INFO

Keywords:

Human action recognition
Slow feature analysis
Subspace-based methods

ABSTRACT

This paper proposes a new video representation for subspace-based action recognition. Traditional subspace-based methods represent a video as a subspace by applying principal component analysis (PCA) to its frames. However, this subspace might lead to an imprecise representation of actions, as PCA loses the temporal information of frames. Therefore, we introduce the slow feature subspace based on the slow feature analysis (SFA). SFA extracts a set of slow features of an input video by projecting the video frames onto the weight vectors that minimize the data variance over time. Motivated by these properties, several methods based on SFA were proposed for action recognition. However, they do not explicitly consider the distribution of the slow features, which represents essential action information. Therefore, our key idea is to capture this distribution through a low-dimensional subspace called slow feature subspace. Our subspace is generated by applying PCA to several weight vectors corresponding to the slowest components obtained by SFA. Our framework replaces the subspaces of several traditional mutual subspace methods with our slow feature subspace to improve their results in action recognition. This approach transforms the comparison between two videos into the comparison between two slow feature subspaces using the canonical angles between them, avoiding vector concatenation and data aggregation. The effectiveness of our framework is demonstrated through extensive experiments with various datasets. Our results show that our slow feature subspace can improve the traditional subspace-based methods and achieve competitive performance compared to different methods, including state-of-the-art neural networks.

1. Introduction

Subspace-based methods have gained renewed attention due to their ability to generalize data distribution even when few samples are available, reduce memory consumption and require less computational resources (Cheok et al., 2019; Fukui, 2020; Fukui & Maki, 2015; Fukui et al., 2006; Fukui & Yamaguchi, 2005; Lu et al., 2017; Sakano & Mukawa, 2000; Yamaguchi et al., 1998). These advantages make the subspace-based methods suitable for several action recognition tasks that run under constraints of data, memory, and processing power (Gatto et al., 2017). In such context, they represent an alternative to more recent neural network-based models that require massive amounts of data and long training hours to achieve high classification accuracy.

Subspace-based methods represent a video as a low-dimensional linear subspace in a high-dimensional feature space modeled from the set of its frames. To obtain the basis vectors of such subspace, principal component analysis (PCA) is usually employed. In these methods, we

compare two subspaces generated from different videos by using the multiple canonical angles between them as the similarity. While PCA can effectively represent the distribution of the frames in terms of spatial variance, it discards the temporal relationship between the frames. This implies that the subspace-based methods cannot deal with more complicated actions in which temporal information plays an essential role.

To address the above problem, several subspace-based methods build a new time representation for the data, which can, for example, be based on Hankel matrices (Gatto et al., 2017) or time elastic (TE) features (Suryanto et al., 2016). Although these approaches achieve competitive results, both lead to high memory consumption due to vector concatenation. Therefore, their representations are often limited to a few frames, affecting their recognition ability as the method will poorly represent the sequence.

Alternatively, the slow feature analysis (SFA) (Wiskott, 1999; Wiskott & Sejnowski, 2002) can represent a video sequence without

* Corresponding author.

E-mail addresses: suzana@cvlab.cs.tsukuba.ac.jp (S.R.A. Belezã), kidoshimomoto.e@aist.go.jp (E.K. Shimomoto), lincon.souza@aist.go.jp (L.S. Souza), kfukui@cs.tsukuba.ac.jp (K. Fukui).

<https://doi.org/10.1016/j.mlwa.2023.100493>

Received 9 February 2023; Received in revised form 9 August 2023; Accepted 20 August 2023

Available online 22 August 2023

2666-8270/© 2023 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

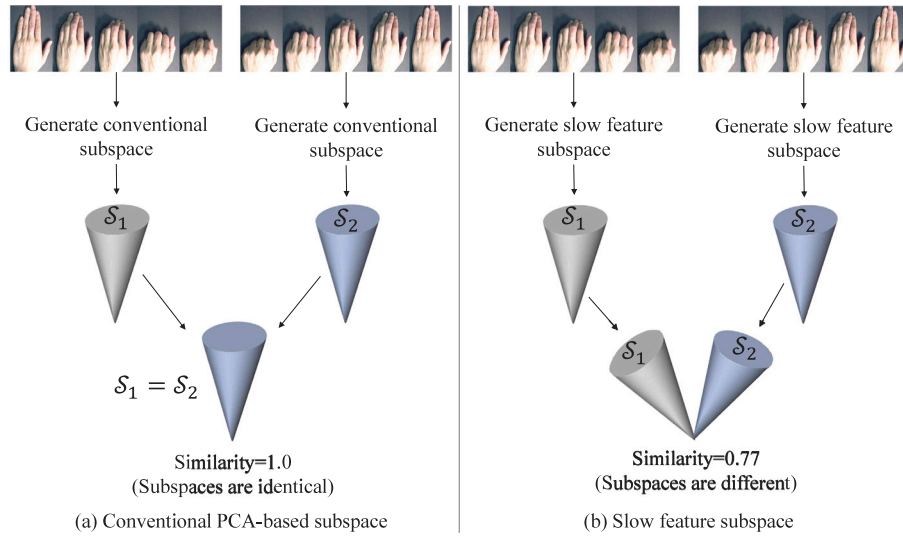


Fig. 1. Action video representation using conventional PCA-based and slow feature subspaces. This example considers two video samples: one for “close hand” and another for “open hand”, where the “open hand” sample contains the reversed frames of the “close hand” video. In (a), the conventional PCA-based subspaces created for each video were identical (similarity = 1.0). In contrast, in (b), the slow feature subspaces are different, as the similarity score was 0.77. This example demonstrates that, unlike the conventional subspaces, slow feature subspaces can effectively distinguish time-related structures even when the videos have the same frames in reversed order.

relying on such vector concatenations. This method connects to the general learning process of the visual cortex (Berkes & Wiskott, 2005; Franzius et al., 2007) as SFA captures the slowly varying characteristics of a sequence over time. These slowly varying characteristics, or slow features, are computed by projecting the original data onto the weight vectors that minimize data variance over time. Thus, SFA builds a compact time representation using low computational requirements, independent of the number of frames of the input video.

Such advantages have made SFA a useful action representation method used in several learning frameworks. However, these frameworks utilize aggregation functions, such as average, sum, and concatenation, to create a single vector representation from the slow features, or weight vectors (Kobayashi, 2017; Sun et al., 2014; Zhang & Tao, 2012). These approaches disregard the distribution of slow feature vectors, which can result in a loss of valuable information to solve our task. To overcome this limitation, one approach is to represent the distribution of slow feature vectors as a low-dimensional subspace using PCA. In this case, to build the subspace, we first compute the weight vectors, extract the slow features by projecting the data onto these vectors, and then apply PCA to the slow features. However, this approach may not be optimal since essential temporal information of the action is rather concentrated in the weight vectors that produce the slow features from the original data (Kobayashi, 2017). Besides, it may involve unnecessary operations since we can build the slow feature subspace directly from the weight vectors.

As the weight vectors are used to obtain the slow features from the original data, it suggests that the weight vectors also hold essential temporal information of the action (Kobayashi, 2017). In fact, the effectiveness of the weight vectors has been discussed in action recognition by Kobayashi (2017), although the method considered only the first weight vector corresponding to the slowest change.

Therefore, in this paper, we focus on weight vectors rather than slow features, where the weight vectors are automatically obtained without additional cost in the process of SFA. Our key idea is to introduce the new geometrical concept of slow feature subspace (SFS), spanned by several weight vectors corresponding to the slowest varying characteristics. According to the above mechanism, we can expect the slow feature subspace to indirectly capture the structure of slow features’ distribution and work as a powerful video representation.

Fig. 1 shows a comparison between the conventional PCA-based subspace (Fig. 1.a) and our SFS (Fig. 1.b). In this example, we compare

two videos: one for “close hand” and another for “open hand”. The “open hand” sample contains the same frames as “close hand” video, but in reversed order. Additionally, the subspace similarity is employed as a metric to quantify the difference between subspaces. A higher similarity score indicates greater equivalence, with 1.0 representing identical subspaces. In contrast, a similarity score smaller than 1.0 indicates that the subspaces are different. As shown in Fig. 1.a, conventional subspaces created from different videos based on the same frames have a similarity of 1.0, which means these subspaces are equal. On the other hand, Fig. 1.b illustrates that the slow feature subspaces are different, with a similarity score of 0.77. Unlike conventional subspaces, our slow feature subspaces can distinguish sequential differences even when the videos have the same frames in reversed order.

Moreover, we can also compare two slow feature subspaces generated from two videos with different number of frames by using multiple canonical angles between them, thereby avoiding the drawbacks of naive operations such as concatenation and data aggregation. Moreover, with our video representation, we avoid extra computation due to projecting data on the weight vectors to extract the slow features.

The proposed slow feature subspace is a new video representation based on SFA that can replace the naive PCA-based subspace in the traditional subspace-based methods. In this paper, our objective is to incorporate our video representation into the mutual subspace method (MSM) (Maeda & Watanabe, 1985), the kernel MSM (KMSM) (Sakano & Mukawa, 2000), the constraint MSM (CMSM) (Fukui & Maki, 2015; Fukui & Yamaguchi, 2005) and the kernel constraint MSM (KCMSM) (Fukui & Maki, 2015) for effective action recognition. In these methods, we compare two slow feature subspaces generated from different videos by the multiple canonical angles between them.

We perform extensive experiments on five widely used action recognition datasets, namely the Cambridge handshape (Kim et al., 2007), KTH action (Schuldt et al., 2004), the Chinese sign language dataset (Liu et al., 2016; Pu, Zhou, Li, 2016; Pu, Zhou, Zhang et al., 2016; Zhang et al., 2016), UCF Sports (Rodriguez et al., 2008; Soomro & Zamir, 2014) and the UCF101 (Soomro et al., 2012). In our experiments, we compare the subspace-based methods using the proposed SFS representation with various types of methods, including the traditional subspace-based methods and state-of-the-art neural networks. The results showed that our video representation significantly improved the performance of the subspace-based methods, achieving competitive results with state-of-the-art methods. Additionally, our results showed

that SFS modeled from pre-trained deep features along with the mutual subspace methods can perform better than the fine-tuned versions of the deep models, suggesting that SFS can be an alternative to fine-tuning.

Concretely, our main contributions are:

1. We introduce a novel video representation called slow feature subspace (SFS), which captures the temporal information among video frames by exploring the geometrical structure of weight vectors obtained through the slow feature analysis.
2. We replace the traditional subspace of four subspace-based methods, namely MSM, KMSM, CMSM and KCMSM, with our SFS and demonstrate that our subspace improves their performance in action recognition.
3. We also show the effectiveness of comparing two slow feature subspaces using multiple canonical angles, facilitating the comparison between videos regardless of their lengths.
4. We demonstrate the advantages of the subspace-based methods and SFS combination over various traditional and modern methods for action recognition through extensive experiments conducted on five public databases.

Although some ideas in this paper have appeared in earlier publication (Beleza & Fukui, 2021) in preliminary form, the present paper contains a more in-depth analysis of the slow feature subspace and its incorporation into the traditional subspace-based methods. Our experimental analysis includes extensive parameter study and comparison with other existing time representations. These time representations include techniques for subspace-based classification, SFA-based approaches, and hand-crafted and deep-learning features.

The rest of this paper is organized as follows. In Section 2, we present the related work regarding video representation and subspace-based methods. After that, we explain the background knowledge on which we build our method in Section 3. Afterward, we introduce our slow feature subspace and framework in Section 4. Next, the experiment results are shown and discussed in Section 5. Finally, we present our final considerations and future work in Section 6.

2. Related work

Action recognition is an important computer vision task with several real-life applications, such as surveillance, human-machine interaction, assistive technologies, among others (Aslan et al., 2020; Sun et al., 2014; Zhang & Tao, 2012). To deal with these tasks, many works exploit neural networks with 3D convolutional layers, e.g., 3DCNNs (Carreira & Zisserman, 2017; Hara et al., 2018; Wang et al., 2018), and long short-term memory (LSTM) aggregated to the default recurrent neural network (RNN) architecture (Donahue et al., 2015; Ng et al., 2015). More recently, transformers have also been employed, with the development of the vision transformer (ViT) (Dosovitskiy et al., 2020) and the video visual transformer (ViViT) (Arnab et al., 2021) concept. While these deep learning-based methods have achieved state-of-the-art results, their performance is highly dependent on the amount of data available, requiring massive amounts of data to train appropriately.

An alternative to such data-hungry methods is the traditional subspace-based methods. They have been extensively used in object recognition (Fukui & Maki, 2015; Fukui et al., 2006; Fukui & Yamaguchi, 2005; Sogi et al., 2018) and in more complex tasks such as action recognition (Sakai et al., 2019; Tanaka et al., 2016). These methods represent sets of images that share a common pattern, such as multi-view images or video frames, as low-dimensional linear subspaces in high-dimensional feature space by applying the principal components analysis (PCA) to the image set. Such representation can effectively generalize the distribution of the features in the set, even when there are little data available.

Further work also showed these methods to be compatible with richer representations extracted from convolutional neural networks

(CNNs), achieving results comparable to more complex architectures (Fukui et al., 2023; Sakai et al., 2019; Sogi et al., 2018). When the subspaces are generated using a transfer learning (e.g., pre-trained features) strategy, the framework can resemble few-shot learning approaches (Li et al., 2022; Wang et al., 2022) for action recognition. However, subspace-based methods are flexible and can learn from several types of data, ranging from sets of raw images to deep features. Therefore, they are not limited to a transfer learning framework.

However, a significant limitation of the traditional subspace-based methods is that they rely on PCA to model the subspaces. As PCA ignores the temporal relation among frames in a video, it fails to adequately capture temporal information, making it insufficient for accurately representing video data.

Aiming at solving this issue, other methods were created to represent the temporal relation of the sequence data along with the subspace representation. The randomized time warping (RTW) (Suryanto et al., 2016), an extension of the dynamic time warping, modeled sequence hypothesis subspaces based on a set of time elastic (TE) features. Extending this work, the RTW with enhanced Grassmann discriminant analysis (RTW+eGDA) (Souza et al., 2020) was proposed, further improving the discriminative capabilities of the method. In parallel, subspaces were also modeled from Hankel matrices (Gatto et al., 2017), capturing spacial and temporal gesture information.

These methods have significantly improved gesture and motion recognition compared to the traditional subspace formulation. However, they suffer from high-computational costs, as both rely on concatenating feature vectors to ensure temporal order, with a significant increase in memory consumption as higher dimensionality feature vectors are used. Such memory consumption limits these methods to explore the temporal relation among very few frames and is especially harmful to the Hankel matrix-based approach, as it works with consecutive ordered features. Ultimately, this issue leads to poor long-term temporal dependency representation.

Without depending on vector concatenations, the slow feature analysis (SFA) (Wiskott, 1999; Wiskott & Sejnowski, 2002) expresses the input sequence based on the slowness principle, which comes from the general learning process of the visual perception (Berkes & Wiskott, 2005; Franzius et al., 2007). The slowness principle states that while the primary sensory signals, such as in the human eye's retina, change on a faster timescale, our environment changes slowly, and therefore, the true representation of the environment should vary on a slow timescale. Following this principle, SFA takes an input video and computes a set of weight vectors used to extract the slowest varying elements of the time series, the slow features. These weight vectors and slow features form compact data representations that do not depend on the number of frames in the video and explore long temporal dependencies.

Such advantages have made SFA a valuable method for action recognition. The accumulated squared derivative feature (ASD) (Zhang & Tao, 2012) was the first work to apply SFA to human action recognition, extracting multiple local cuboids from training videos and applying SFA to concatenated patches to represent temporal information. They then compute squared first-order temporal derivatives and accumulate them over all cuboids through an aggregation function, i.e., the sum. DL-SFA (Sun et al., 2014) also follows the same cuboid strategy for SFA inputs, but the generated weight vectors are used as a 3D convolution filter in a deep neural network. More recently, a regularized version of SFA was proposed, the PCA-SFA (Kobayashi, 2017), which solved the instabilities of SFA in small-sample size settings. However, it relies only on the first weight vector generated by SFA, not exploring all temporal information given by the feature space distribution.

Furthermore, a common issue of these SFA-based methods is that they summarize the rich temporal representation given from SFA by concatenating slow features into a single vector representation, disregarding their distribution and leading to information loss.

Given this context, our proposed slow feature subspace can effectively integrate the temporal information extracted by SFA into the subspace representation and consider the feature space distribution. Moreover, by representing videos as slow feature subspaces, we can take advantage of the vast literature on general-purpose classification methods that can work directly with the subspace representation without resorting to feature aggregation strategies.

3. Background knowledge

This section briefly explains the fundamental knowledge on which our slow feature subspace and the proposed framework are based. We first introduce the slow feature analysis and its regularized version, and then discuss the mutual subspace methods to which our subspace was integrated.

3.1. Slow feature analysis

The slow feature analysis (SFA) is an unsupervised method that extracts slow features to represent a rapid varying input sequence into a smoother sequence. Below, we describe the slow feature extraction mechanism for video action recognition.

Consider a video, represented as a set of feature vectors extracted over time as $X = [x_1, \dots, x_T]$, where $x_t \in \mathbb{R}^d$, t is the time index, T is the total number of features and d is the dimension of the feature vector. To represent X , SFA calculates a set of weight vectors $w = [w_1, \dots, w_T]$, where $w_i \in \mathbb{R}^d$, which is a mapping function between the original features x_t and the output signal $y_t = w^T x_t + b$.

The slow features y_t smoothly change through the sequence under the following three constraints: zero mean $\frac{1}{T} \sum_{t=1}^T y_t = 0$, unit variance $\frac{1}{T} \sum_{t=1}^T y_t^2 = 1$ and decorrelation $\frac{1}{T} \sum_{t=1}^T y_t^{(i)} y_t^{(j)} = 0 \forall i < j$, where $y_t^{(i)} = w_i^T x_t + b_i$ is the i th output value by the i th projection w_i . Under these conditions, the generalized eigenvalue problem of SFA is formulated according to the following equation:

$$\left(\sum_t \dot{x}_t \dot{x}_t^T \right) w = \lambda \left(\sum_t (x_t - \mu)(x_t - \mu)^T \right) w, \quad (1)$$

where $\mu = \frac{1}{T} \sum_t x_t$ is the mean and $\dot{x}_t = x_{t+1} - x_t$ is the differential vector on t responsible for exploiting the time structure of the sample. Ultimately, this optimization problem minimizes the data variance over time, which makes the output signal smoother.

Generally, only the slow features are used for action classification (Fan et al., 2019; Shan et al., 2014; Zhang & Tao, 2012). In these frameworks, dimensionality reduction or vectorization mechanisms are employed to reduce the number of slow feature vectors to a single vector, which is then used by a machine learning algorithm for classification. While this strategy allows slow features to be used along with traditional machine learning methods, such as SVM, this vectorization ignores the structure information represented by the space spanned by the weight vectors.

3.2. Regularized slow feature analysis

In the action recognition task, the datasets frequently have video samples where $T < d$, which affects the SFA eigenvalue problem of Eq. (1). In other words, SFA suffers overfitting when the number of video frames (T) is smaller than the dimension (d) of each frame. To deal with this small sample size (SSS) problem, PCA was integrated into the original SFA formulation, creating the PCA-SFA (Kobayashi, 2017) method. The PCA-SFA regularizes the weight vectors w to follow the PCA subspace of the feature sequence to extract the temporal information via SFA effectively.

Formally, the eigenvalue problems of PCA and SFA are equivalent to minimization problems that share the same denominator (Kobayashi, 2017). Therefore, both methods can be merged using Eq. (2).

$$\min_w \frac{w^T (\eta I + \sum_t \dot{x}_t \dot{x}_t^T) w}{w^T \{ \sum_t (x_t - \mu)(x_t - \mu)^T \} w} \Leftrightarrow \left(\eta I + \sum_t \dot{x}_t \dot{x}_t^T \right) w = \lambda \left(\sum_t (x_t - \mu)(x_t - \mu)^T \right) w, \quad (2)$$

where η is a balancing parameter between PCA and SFA, also referred to as the PCA contribution coefficient, that is defined as $\eta = \frac{1}{T} \sum_t \|\dot{x}_t\|_2^2$. This parameter becomes higher for the smaller-sized samples and lower for the larger-sized ones.

The PCA-SFA is efficiently computed in the case of SSS as follows. First, the features are centered by using $X - \mu \mathbf{1}^T$, where $X \in \mathbb{R}^{d \times T}$ ($T \ll d$) with the features vectors x_t as its columns. Second, singular vector decomposition (SVD) is applied to the centered sequence $X - \mu \mathbf{1}^T = U \Sigma V^T$ where $U \in \mathbb{R}^{d \times r}$, $\Sigma \in \mathbb{R}^{r \times r}$, $V \in \mathbb{R}^{T \times r}$ and $r(< T)$ indicates the rank, usually $r = T - 1$. Then, Eq. (2) can be rewritten as Eq. (3) after reparameterizing the weight vectors as $w = U \Sigma^{-1} \alpha$.

$$\Sigma^{-1} U^T \left(\eta I + \sum_t \dot{x}_t \dot{x}_t^T \right) U \Sigma^{-1} \alpha = \lambda \alpha \Leftrightarrow \left(\eta \Sigma^{-2} + \Sigma^{-1} U^T \sum_t \dot{x}_t \dot{x}_t^T U \Sigma^{-1} \right) \alpha = \lambda \alpha. \quad (3)$$

By solving the eigenvalue problem above, we can find the eigenvector α and calculate w . After that, w is sorted in ascending order following the eigenvalues in λ . Hence, the first weight vector w_1 will correspond to the 1st order component and the last one to the T th order one.

After calculating the weight vectors, Kobayashi (2017) also proposes a video descriptor based on SFA. The descriptor consists of the concatenation of the mean of frames x_t and the first weight vector w_1 (corresponding to the smallest eigenvalue). Concretely, the descriptor is computed as follows.

The mean is simply calculated as $\mu = \frac{1}{T} \sum_{t=1}^T x_t$. The directional sign of the projection vector w_1 is corrected with respect to time correlation. That is, it is aligned such that the slow features $y_t = w_1^T (x_t - \mu)$ exhibit a positive correlation with the time index t . The alignment can be written as:

$$\hat{w}_1 = \begin{cases} w_1 & \text{corr}(w_1^T x_t, t) \geq 0 \\ -w_1 & \text{corr}(w_1^T x_t, t) < 0, \end{cases} \quad (4)$$

where corr corresponds to the correlation coefficient between two sequences in time.

The video descriptor is then defined as:

$$v = \left[\frac{\mu^T}{\|\mu\|_2}, \frac{\hat{w}_1^T}{\|\hat{w}_1\|_2} \right]^T \in \mathbb{R}^{2d}. \quad (5)$$

To the best of our knowledge, PCA-SFA is the first work that directly employs a weight vector as a descriptor, in contrast to previous works that leverage SFA to extract slow features, paying less attention to the weight vectors. However, we should note that they employ only the first weight vector, corresponding to the smallest eigenvalue. As such, this descriptor is not able to capture more complete information regarding the data distribution, whereas our proposed method leverages multiple weight vectors. In Section 5.4, we empirically compare this descriptor and our proposed method.

3.3. Mutual subspace methods

The subspace-based methods constitute a line of works developed since the 70's targeting the tasks of image classification. The main idea is to model images of an entity, e.g., images of the same person, as a low-dimensional linear subspace in a high-dimensional feature

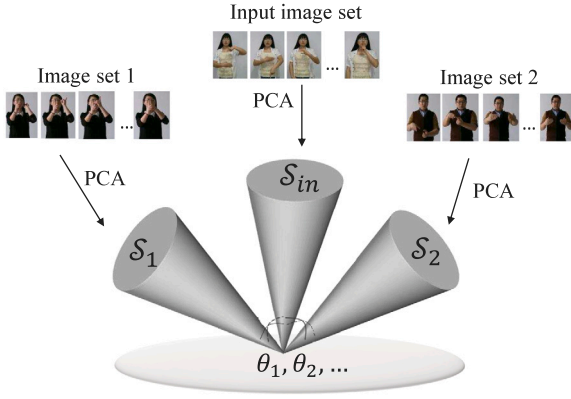


Fig. 2. Conceptual diagram of MSM. On MSM, each image set is represented as a subspace after applying PCA. The canonical angles formed between two subspaces are used to measure their similarity. The input subspace S_{in} is then assigned to the class with the highest similarity.

space, and then manipulate these subspaces obeying their mathematical properties.

Modeling entities as subspaces comes from the empirical observation that images that share a common pattern tend to form a compact cluster in a high-dimensional feature space (Iijima et al., 1974; Watanabe & Pakvasa, 1973). While there are several ways to obtain the basis vectors of such subspace, most of the subspace-based methods use PCA. PCA retains most of the variability of the features in a set of images, effectively and compactly representing the main patterns present in the set.

Formally, consider a set of T feature vectors $\{x_i\}_{i=1}^T$ representing an entity, e.g., multi-view images of a person, stacked as the columns of the matrix $X \in \mathbb{R}^{d \times T}$, where d is the dimension of the original feature space. We apply PCA without data centering to X to model a subspace from this set of features. The orthonormal basis vectors of the N -dimensional subspace S are obtained as the eigenvectors with the N largest eigenvalues $\{\lambda_i\}_{i=1}^N$ of the matrix $R = XX^T$, where $N < T$. The entity is finally represented as $S = [\Phi_1 \dots \Phi_N] \in \mathbb{R}^{d \times N}$, which has the corresponding orthonormal basis vectors as its column vectors.

Among the vast literature on subspace-based methods, our work focus on a specific family: the mutual subspace methods. Consider a training dataset, denoted as $\{(X_i, y_i)\}_{i=1}^m$, where $X_i \in \mathbb{R}^{d \times T}$ is a set of T features representing a sample, $y_i \in (1, 2, \dots, C)$ is its class label, C is the number of classes, and m is the number of sets in the dataset. Additionally, X_{in} represents an input set. The mutual subspace methods model sets from each class into class subspaces $\{S_c\}_{c=1}^C$ and the input set as input subspace S_{in} . The classification of the input set is then performed by assigning it to the closest class subspace, based on the subspace similarity.

In the remaining of this section, we will discuss the main mutual subspace methods used in our work, namely the mutual subspace method (MSM) (Maeda & Watanabe, 1985) and its discriminative version, the constrained MSM (CMSM) (Fukui & Maki, 2015; Fukui & Yamaguchi, 2005), as well as their non-linear variants, KMSM (Sakano & Mukawa, 2000) and KCMSM (Fukui & Maki, 2015; Fukui & Yamaguchi, 2005).

3.3.1. Mutual Subspace Method (MSM)

The mutual subspace method (MSM) (Maeda & Watanabe, 1985) is the most fundamental method within the mutual subspace methods used in this work. Fig. 2 shows its conceptual diagram. Given an input subspace S_{in} and class subspaces $\{S_c\}$, it assigns the input subspace to the class of the closest class subspace based on the subspace similarity.

The subspace similarity is calculated based on the canonical angles. Consider an N_{in} -dimensional input subspace S_{in} and a N_c -dimensional class subspace S_c in a d -dimensional vector space, where $N_{in} \leq N_c$. The canonical angles $\{0 \leq \theta_1, \dots, \theta_{N_{in}} \leq \frac{\pi}{2}\}$ between the S_{in} and S_c are recursively defined as follows (Afriat, 1957; Hotelling, 1992):

$$\cos \theta_i = \max_{u \in S_{in}} \max_{v \in S_c} u^T v = u_i^T v_i, \quad (6)$$

$$s.t. \|u_i\|_2 = \|v_i\|_2 = 1, u_i^T u_j = v_i^T v_j = 0, i \neq j,$$

where u_i and v_i are the canonical vectors forming the i th smallest canonical angle θ_i between S_{in} and S_c . The j th canonical angle θ_j is the smallest angle in the direction orthogonal to the canonical angles $\{\theta_k\}_{k=1}^{j-1}$.

These canonical angles can be easily calculated by applying the SVD to $S_{in}^T S_c$, where $S_{in} \in \mathbb{R}^{d \times N_{in}}$ represents the basis vectors of S_{in} and $S_c \in \mathbb{R}^{d \times N_c}$, the basis vectors of S_c . The geometric similarity between two subspaces S_{in} and S_c is then defined as in Eq. (7).

$$\text{sim}(S_c, S_{in}) = \frac{1}{N_1} \sum_{i=1}^{N_1} \cos^2 \theta_i. \quad (7)$$

However, the SVD of $S_{in}^T S_c$ is computationally expensive. Therefore, as in Souza et al. (2023), we leverage the spectral definition of the Frobenius norm to compute the similarity as in Eq. (8) without solving the eigenvalue problem of $S_{in}^T S_c$:

$$\text{sim}(S_c, S_{in}) = \frac{1}{N_1} \text{tr}(S_c^T S_{in} S_{in}^T S_c). \quad (8)$$

A special case of MSM is when the input is reduced to be a vector x_{in} rather than a subspace S_{in} . In this case, MSM is reduced to what is referred to simply as subspace method (SM), first proposed by Iijima as the multiple similarity method (Iijima et al., 1974) and by Watanabe as CLAFIC (Watanabe, 1967). The similarity simplifies as follows, becoming a matching between a subspace S_c and normalized vector x_{in} :

$$\text{sim}(S_c, x_{in}) = x_{in}^T S_c x_{in}, \quad (9)$$

which is equivalent to the squared cosine of the angle θ between x_{in} and its projection onto the subspace S_c . SM is employed as a baseline in the experiment in Section 5.4 where only one vector descriptor is available.

To handle non-linear data distributions, the kernel mutual subspace method (KMSM) (Sakano & Mukawa, 2000) performs MSM in a reproducing kernel Hilbert space. In this space, computing the projections directly is challenging as the dimensions are extremely high. Therefore, a kernel PCA (KPCA) (Schölkopf et al., 1998) is performed by a kernel trick. The Gaussian radial basis function is a commonly used kernel functions defined as $k(x_i, x_j)$ in Eq. (10), where x_i and x_j belong to the training input, and σ^2 is the kernel bandwidth parameter.

$$k(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{\sigma^2}\right). \quad (10)$$

3.3.2. Constrained and kernel constrained mutual subspace methods

The constrained mutual subspace method (CMSM) (Fukui & Maki, 2015; Fukui et al., 2023, 2006; Fukui & Yamaguchi, 2005) is a discriminative version of MSM. Fig. 3 shows the conceptual diagram of CMSM. After obtaining the class subspaces $\{S_c\}_{c=1}^C$, we model the generalized difference subspace (GDS), $\mathcal{G}$. Then, all class and input subspaces are projected onto the GDS. Through this projection, common information among class subspaces is discarded. Classification then is performed as in MSM. In essence, the GDS $\mathcal{G}$ is an extension of the difference subspace between two subspaces, which itself is a generalization of the difference vector between two vectors (Fukui & Maki, 2015; Fukui et al., 2006; Souza et al., 2020).

Given $C(\geq 2)$ N_c -dimensional class subspaces, $\{S_c\}_{c=1}^C$, the GDS $\mathcal{G}$ can be defined as the subspace generated by the removal of the

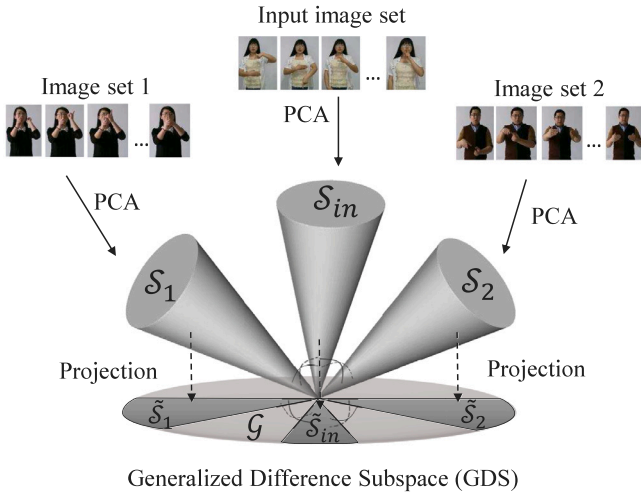


Fig. 3. Conceptual diagram of CMSM. On CMSM, the class subspaces $\{S_c\}_{c=1}^C$ and the input subspace S_{in} are projected onto the GDS $\mathcal{G}$. This operation computes the $\{\tilde{S}_c\}_{c=1}^C$, which discards interclass shared characteristics, and $\tilde{S}_{in}$ that represents only intrinsic characteristics of the input sequence. After that, the classification is done by using the subspace similarity between the class subspaces $\{\tilde{S}_c\}_{c=1}^C$ and input subspace $\tilde{S}_{in}$.

principal component subspace $\mathcal{M}$ of all class subspaces from the sum subspace $\mathcal{Q}$. Therefore, the GDS projection removes shared characteristics among class subspaces and increases the orthogonal degree between the subspaces which can improve the classification based on canonical angles.

To obtain the GDS $\mathcal{G}$ basis vectors, we first compute the sum of the projection matrices of the class subspaces, $\mathbf{P} = \sum_{c=1}^C \mathbf{S}_c \mathbf{S}_c^T$, where $\mathbf{S}_c$ contains basis vectors of the c th class subspace. The GDS $\mathcal{G}$ is spanned by N_g smallest eigenvectors of the sum matrix $\mathbf{P}$. The optimal dimension N_g of the GDS is often determined empirically by observing the orthogonal degree between the classes.

Moreover, to deal with non-linear classification and keep the advantages mentioned, CMSM was extended to the kernel constrained mutual subspace method (KCMSM) (Fukui & Maki, 2015; Fukui et al., 2006) by using a kernel trick to compute the kernel GDS. For further details, we refer the readers to the original papers (Fukui & Maki, 2015; Fukui et al., 2023, 2006).

4. Proposed approach and framework

In this section, we describe the proposed method. First, we introduce our slow feature subspace, and then explain how we employ it in a subspace-based recognition framework. Finally, we also discuss the time and space complexities of our approaches.

4.1. Slow feature subspace to represent temporal data

To compute the proposed slow feature subspace (SFS), we first extract the PCA-SFA weight vectors and then obtain an orthonormal basis of these vectors, which spans the SFS. This process will be described in the following.

As in PCA-SFA, the sequence $\mathbf{X} \in \mathbb{R}^{d \times T}$ are centered by $\mathbf{X} - \mu \mathbf{1}^T$ and then decomposed by singular value decomposition (SVD), i.e.:

$$\mathbf{X} - \mu \mathbf{1}^T = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T, \quad (11)$$

where $\mathbf{U} \in \mathbb{R}^{d \times r}$, $\mathbf{\Sigma} \in \mathbb{R}^{r \times r}$, $\mathbf{V} \in \mathbb{R}^{T \times r}$ and $r(< T)$ indicates the rank, usually $r = T - 1$.

Then, we solve the following eigenproblem, which is equivalent to Eq. (3):

$$\left(\eta \mathbf{\Sigma}^{-2} + \mathbf{\Sigma}^{-1} \mathbf{U}^T \dot{\mathbf{X}} \dot{\mathbf{X}}^T \mathbf{U} \mathbf{\Sigma}^{-1} \right) \boldsymbol{\alpha} = \lambda \boldsymbol{\alpha}, \quad (12)$$

where $\dot{\mathbf{X}} \in \mathbb{R}^{d \times T}$ is a matrix where columns are the differentials $\dot{\mathbf{x}}_t = \mathbf{x}_{t+1} - \mathbf{x}_t$. Specially when $t = T$, $\dot{\mathbf{x}}_t = -\mathbf{x}_t$.

We then sort the eigenvectors $\boldsymbol{\alpha}_i$ in ascending order according to the eigenvalues λ_i and compute the weight vectors $\mathbf{w}_i = \mathbf{U} \mathbf{\Sigma}^{-1} \boldsymbol{\alpha}_i$. In our proposed method, we retain the slowest $p \in \{1, \dots, r\}$ weight vectors.

Next, we create a video descriptor $\mathbf{W} \in \mathbb{R}^{d \times p}$ and then generate our slow feature subspace $\mathcal{S}$ via SVD of $\mathbf{W}$:

$$\mathbf{W} = \begin{bmatrix} \mathbf{w}_1 & \dots & \mathbf{w}_p \\ \|\mathbf{w}_1\|_2 & \dots & \|\mathbf{w}_p\|_2 \end{bmatrix} = \mathbf{J} \mathbf{\Theta} \mathbf{Q}^T. \quad (13)$$

The matrix $\mathbf{W}$ is composed of the normalized weight vectors, $\mathbf{J} \in \mathbb{R}^{d \times p}$ and $\mathbf{Q} \in \mathbb{R}^{p \times p}$ are the orthogonal matrices of left and right singular vectors, and $\mathbf{\Theta} \in \mathbb{R}^{p \times p}$ is a diagonal matrix of singular values. Additionally, the singular vectors are sorted according to the singular values.

The basis $\mathcal{S} \in \mathbb{R}^{d \times q}$ of the q -dimensional slow feature space $\mathcal{S}$ is obtained as the first q singular vectors obtained in the eigenvalue problem of Eq. (13), i.e.:

$$\mathcal{S} = \mathbf{J}_q. \quad (14)$$

In this way, we compute our proposed video representation, the slow feature subspace $\mathcal{S}$, which is modeled from the proposed video descriptor $\mathbf{W}$.

Note that our method requires only the computation of the weight vectors, as they minimize the data variation over time and can independently represent a time series (Kobayashi, 2017).

The conceptual diagram of the proposed method is shown in Fig. 4. Given an input video consisting of a sequence $\mathbf{X}$, we first extract a set of weight vectors using the regularized version of SFA, organized as the columns of a video descriptor matrix $\mathbf{W}$. Then, we apply PCA without data centering to $\mathbf{W}$ to obtain the basis vectors that span the slow feature subspace (SFS) of the input video. Likewise, we model the reference subspaces from class samples. Finally, we replace the conventional subspace of the mutual subspace methods (MSM, CMSM, KMSM, KCMSM) with our subspace and perform action recognition. These classifiers can effectively compare two sets of weight vectors by calculating the subspace similarity between the slow feature subspaces.

4.2. Proposed framework for action recognition

In this session, we explain the proposed framework for video action recognition by using our SFS combined with the mutual subspace methods.

Fig. 5 shows the flowchart of our framework, which is divided into training and recognition phases. The training dataset is denoted as $\{(\mathbf{X}_i, y_i)\}_{i=1}^m$, where $\mathbf{X}_i \in \mathbb{R}^{d \times T}$ is a sequence, $y_i \in \{1, 2, \dots, C\}$ is its class label, and m is the number of sequences in the dataset. These input sequences can have different lengths (i.e., different numbers of frames). If raw features (raw image frames) are used, the dimension $d = h \times w$, where h is the height, and w is the width of the frames. If feature extraction is used, such as deep neural network or handcrafted features, d denotes the dimension of such features. To summarize, $\mathbf{X}_i$ is a sequence that can represent a video by vectorizing its raw frames or using deep/handcrafted features.

By using the algorithm proposed in the previous subsection, a video descriptor can be extracted for each sequence, forming a set of training descriptors $\{\mathbf{W}_i\}_{i=1}^m$. The detailed flow to build our slow feature subspace from $\mathbf{W}_i$ is described below.

In the training phase, there are four steps:

1. Features are extracted frame-wise for each video in the training set, resulting in sequences $\{\mathbf{X}_i\}_{i=1}^m$.
2. The video descriptors $\{\mathbf{W}_i\}_{i=1}^m$ are obtained by selecting the p -first weight vectors extracted from the sequence $\{\mathbf{X}_i\}$.

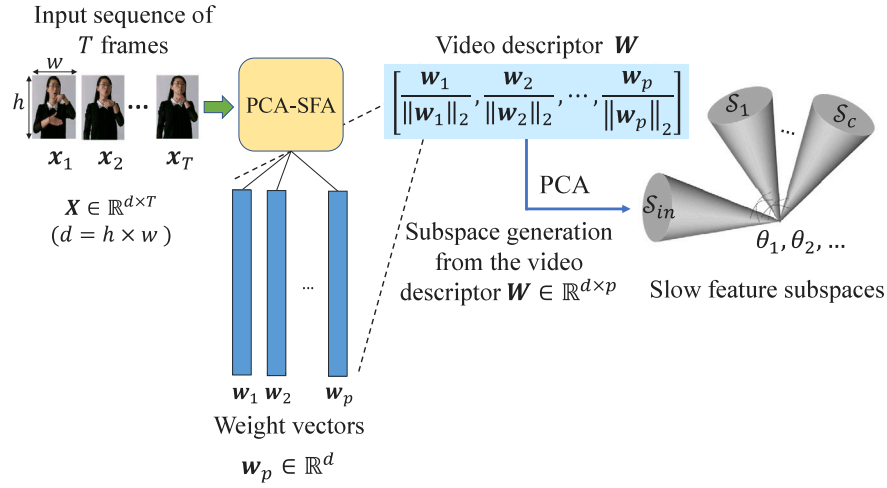


Fig. 4. Conceptual diagram of our proposed method. Given an input sequence X , we first extract p weight vectors w using a variation of PCA-SFA. Then, we model the input slow feature subspace S_{in} by applying PCA without data-centering to W . Analogously, given sequences from different action classes, we model reference slow feature subspaces S_c , where $c = \{1, \dots, C\}$ and C is the number of classes. Action recognition is then performed based on the subspace similarity, calculated using the canonical angles $\{\theta\}$ using one of the MSMs.

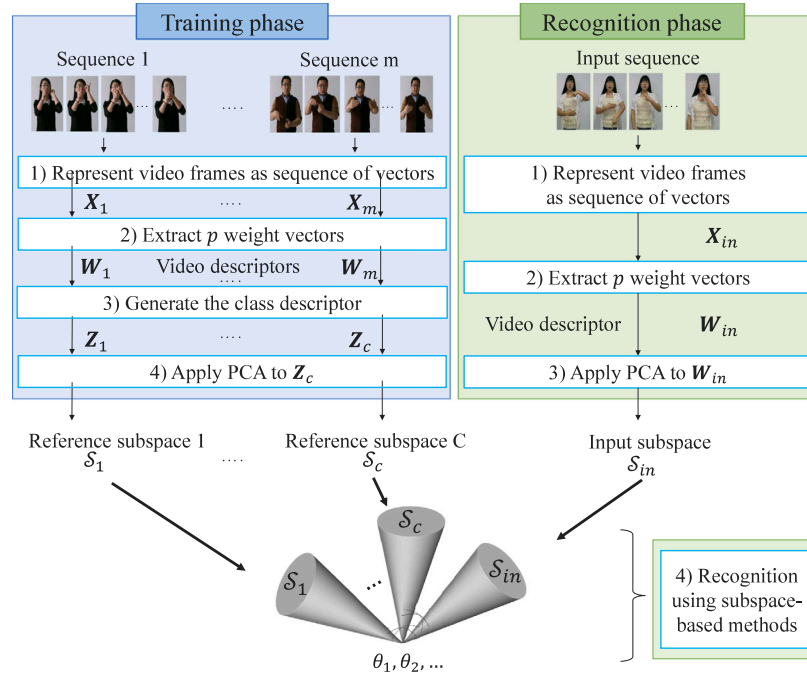


Fig. 5. The proposed framework for action recognition is divided into training and recognition. In the training phase, video descriptors are computed following the weight vectors (basis vectors) of the slow feature space. After that, subspaces are modeled to each class using the normalized video descriptors. In the recognition phase, a subspace is modeled from the input video descriptor and classified using a subspace-based method.

3. Descriptors of the same class are combined into a class descriptor. For a class c , the class descriptor is $Z_c = [w_{i|y_i=c}]$. Note that $c = 1, \dots, C$.
4. A basis S_c of class subspace S_c is computed by applying PCA (or KPCA), without data-centering, to each class descriptor Z_c .

Depending on the mutual subspace-based method used for classification, we can also apply KPCA to build the subspaces. Additionally, CMSM and KCMSM project the class subspaces S_c on the GDS. Next, the recognition phase is divided into:

1. A given input video goes through frame-wise feature extraction, resulting in a sequence X_{in} .

2. The video descriptor W_{in} is obtained from sequence X_{in} , using the proposed algorithm.
3. A basis S_{in} for the input subspace S_{in} is generated by applying PCA (or KPCA), without data-centering to W_{in} .
4. One of the mutual subspace methods (i.e., MSM, CMSM, KMSM, KCMSM) is applied to compare the input subspace S_{in} and class subspaces $\{S_c\}_{c=1}^C$ to assign the video to the class with the highest similarity.

4.3. Computational complexity of proposed methods

This section discusses our time and space complexity in terms of Big O notation. Table 1 summarizes the dominant time and space

Table 1

Comparison of time and space complexities between the traditional MSM and the MSM with our slow feature subspace.

Algorithm	Time complexity	Space complexity
Traditional MSM	$O(Cd(nT)^2)$	$O(CdnT)$
MSM with SFS (ours)	$O(mdT^2)$	$O(md p)$

complexities of the traditional MSM and the MSM with our SFS. The variables used in this table have already been described in this section, except for n , which represents the number of samples per class.

In terms of time complexity, both methods share a common bottleneck related to the SVD utilized within their frameworks. Specifically, for the traditional MSM method, the complexity is referent to the generation of class subspaces in the training phase, which is similar to the third step of our framework discussed in Section 4.2. Similarly, for the MSM with SFS approach, the execution time is largely attributed to solving the eigenvalue problem as described in Eq. (11) for all m samples.

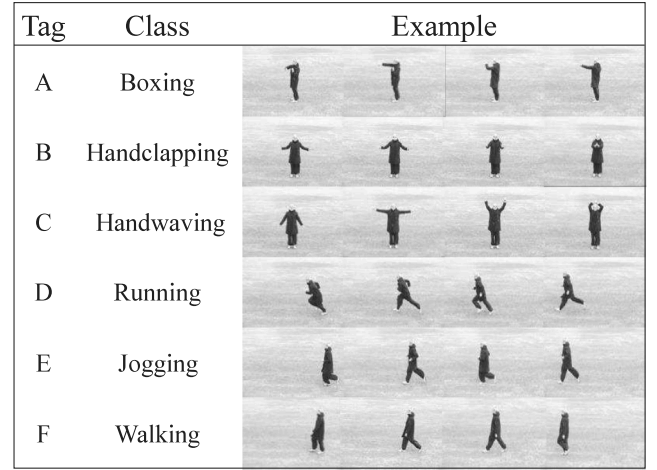
In our analysis, the upper bound of the time complexity for the SVD operation on a $d \times T$ matrix is considered to be $O(dT^2)$ (Batalo et al., 2022). As such, our approaches would require a quadratic execution time at most. To facilitate comparison, we assumed that the total number of samples is given by $m = Cn$, where C is the number of classes and n is the number of samples per class. Under this assumption, the time complexity of the traditional MSM can be expressed as $O(mdnT^2)$. When $n = 1$, the complexities of both approaches are equal. However, when $n > 1$, the time complexity of the traditional MSM becomes higher than our approach with SFS. Similar analysis can be extended to other methods such as CMSM, KMSM, and KCMSM. However, in general, the execution time of these methods might be higher than that of the MSM approach due to additional operations such as GDS and kernel computation.

Regarding space complexity, we consider the generation of class subspaces in the training phase for the original MSM method. In this case, the SVD operation typically requires storing the input data resulting in a $O(dT)$ space complexity. For our method, we consider the space complexity of generating m sets of p weight vectors as described in Eq. (13). Comparing the space complexities, the original MSM method mainly relies on storing the input matrix for the SVD operation, while our method with SFS involves storing the weight vectors of all samples.

When considering $m = Cn$, the space complexity of the traditional MSM can be expressed as $O(mdT)$, whereas our approach with SFS has a space complexity of $O(md p)$. Here, p represents the number of weight vectors, which is typically smaller than T , the dimensionality of the feature space. Comparing the space complexities of the two methods, the incorporation of SFS into the MSM framework requires less space than the original MSM. This reduction in space complexity is attributed to our approach storing the weight vectors instead of the input matrix used in the SVD. The same explanation can be extended to other methods such as CMSM, KMSM, and KCMSM. By incorporating our subspace representation, the overall space complexity of these frameworks is likely to be reduced.

The analysis of time and space complexity reveals that our approach achieves comparable or shorter execution time than traditional methods and also significantly reduces the space complexity of the algorithms. This finding highlights the advantage of incorporating slow feature subspaces into our framework. By leveraging SFS, we can enhance the representation of action videos, leading to improved performance in action recognition tasks. At the same time, we effectively reduce the space complexity of the traditional methods. This reduction in space complexity is particularly beneficial in scenarios where memory resources are limited or when dealing with large datasets.

In summary, incorporating slow feature subspaces in our framework enhances representation and reduces space complexity without compromising execution time. This contributes to the overall efficiency and effectiveness of our method in action recognition tasks.

**Fig. 6.** Examples of KTH action dataset.

5. Experimental results

We assess our slow feature subspace (SFS) performance through extensive experiments in this section. First, we introduce the datasets and the computational environment used in our experiments in Sections 5.1 and 5.2, respectively. Then, in Section 5.3, we study the hyperparameters of our subspace to investigate the ranges in which it performs the best. After that, in Sections 5.4–5.7, we compare our performance to several time representations, SFA-based approaches, and neural networks on action recognition, respectively. Moreover, we implemented the comparison methods following their respective papers, unless their results were already available in an earlier publication. In this case, the papers from which the results were extracted are cited in the section where they are discussed.

5.1. Datasets

In our experiments, we used the following four datasets:

Cambridge Gesture (CMB) (Kim et al., 2007) contains 9 classes of hand gestures. Each class consists of videos recorded in 5 different illuminations, in 10 arbitrary motions, from 2 subjects, which results in 100 samples. In our experiments, each frame was resized to 12×16 pixels in grayscale, resulting in a 192-dimensional vector for each frame.

KTH action (KTH) (Schuldt et al., 2004) has 6 classes of human actions: boxing, handclapping, handwaving, running, jogging and walking, which are exemplified in Fig. 6. These actions were performed by 25 individuals in different scenarios: outdoors, outdoors with zoom variation, outdoors with different clothes, and indoors. The six classes and the four filming conditions have four samples each. In our classification experiments, we resized each frame to 16×16 pixels such that each frame has 256 dimensions.

Chinese sign language dataset (CSL) (Liu et al., 2016; Pu, Zhou, Li, 2016; Pu, Zhou, Zhang et al., 2016; Zhang et al., 2016) consists of 500 signs of the Chinese sign language, each performed by 50 subjects, resulting in 25 000 videos. These hand signs can be formed by one or more hand shapes that can be represented at different speeds depending on the person performing the gesture, posing a much bigger challenge the previous two datasets. Examples from this dataset can be seen in Fig. 7. We randomly sampled 10 signs from the entire dataset with their 50 subjects for our experiments, resulting in a subset of 500 videos. Specially for this dataset, the details of frame dimension will be given in its respective experimental sections.

UCF Sports dataset (UCF) (Rodriguez et al., 2008; Soomro & Zamir, 2014) contains 150 samples of individuals performing sports that are

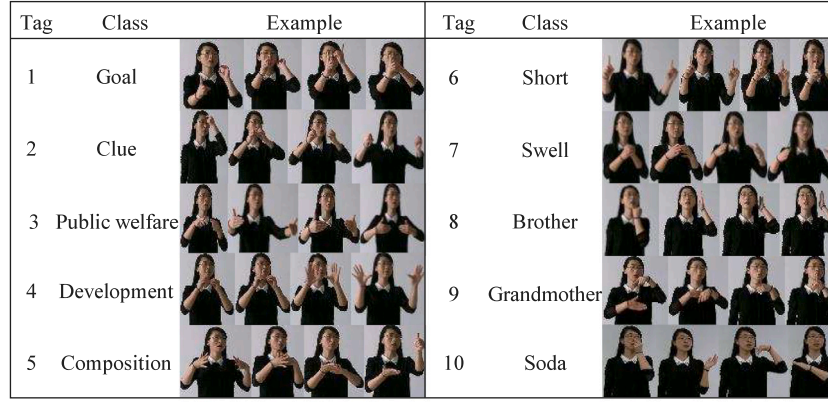


Fig. 7. Examples of the Chinese sign language dataset.

divided into 10 classes: diving, golf swing, kicking, lifting, riding a horse, running, skateboarding, swing-bench, swing-side and walking. The number of frames of each video sample varies from 50 to 70. Besides, the actions were isolated by cropping the frames using the provided annotations. Then, we resized the cropped frames to a 38×24 pixels grayscale image, resulting in a 912-dimensional vector for each frame.

UCF101 (Soomro et al., 2012) has 13 320 realistic action videos collected from Youtube, separated into 101 classes. This dataset contains a variety of camera motions, object appearance and pose, object scale, and illumination conditions, for example. Our experiments report the results in the three train/test splits available for this dataset.

The datasets were chosen based on the complexity of the actions. For instance, the CMB and KTH datasets were selected for preliminary hyperparameter study and comparison of existing subspace representations for action recognition due to their simpler actions. On the other hand, the CSL, UCF Sports, and UCF101 datasets, which contain more complex actions, were included to demonstrate the capability of our proposed approach to represent such samples and to allow comparison of our results with other existing methods.

5.2. Computational environment

We used two different machines to conduct our experiments:

- For all experiments using our SFS and subspace-based methods, we used a machine with Intel® Core™ i9 CPU, and 64 GB of RAM, running Ubuntu 22.04. For the experiments with I3D (Carreira & Zisserman, 2017; Wang et al., 2018), we used the same computer, but with one 10-GB NVIDIA GeForce RTX 3080 GPU.
- Specifically for the experiments with TimeSformer (Bertasius et al., 2021) and Video Swin Transformer (Liu et al., 2022), we used a server machine with Intel® Xeon® E5-2680 v4 CPU and eight 16-GB NVIDIA V100 GPUs.

The experiments with the subspace-based methods were implemented using Matlab 2021b. Moreover, the experiments with the neural networks were developed using python 3.8, PyTorch 1.10, CUDA 11.3, and the MMAAction2 (Contributors, 2020) open-source toolbox.

5.3. Hyperparameters study

The proposed SFS has two hyperparameters: the number of weight vectors (p) and the PCA contribution coefficient (η). In order to better understand the effect of these parameters on performance, we conducted experiments using the Cambridge gesture and KTH action datasets.

Table 2

Evaluation of the methods behavior given the variation of the number of weight vectors (p). The results show that with the increase of the number of weight vectors, the classification accuracy (%) tends, in average, to decrease.

Weight vectors (p)	MSM	KMSM	CMSM	KCMSM	Average
10	82.50	77.40	82.08	81.50	80.87
20	77.90	82.20	78.10	82.50	80.18
30	75.40	78.90	76.00	81.80	78.03

5.3.1. Number of weight vectors

In this experiment, we vary the number of weight vectors p to understand how it influences the methods' performances, and then, we select the number that achieves the best overall performance.

Experimental setup: For this experiment, we used the Cambridge gesture dataset, using all 20 sequences in the normal illumination setting (set 5) for training and the remaining sequences in other illumination settings (sets 1 to 4) for testing. We varied the MSMs hyperparameters, such as the class subspace S_c dimension, ranging from 50 to 80 in intervals of 10, and the input subspace S_{in} dimension, from 2 to 10 in intervals of 2. For the kernel methods, the Gaussian kernel parameter σ^2 ranged from 0.4 to 0.9 in increments of 0.1. For CMSM, the GDS $\mathcal{G}$ dimension varied from 2 to 30 in intervals of 2, while it varied from 400 to 700 in increments of 10 in KCMSM. We report the results for the combination of hyperparameters that led to the best average performance, according to the grid search across all 4 test sets.

Results and discussion: Table 2 presents the classification accuracy obtained by the analyzed methods on the dataset, given different numbers of weight vectors. The last column of this table shows the average performance across classifiers for the same amount of weight vectors. The results show that as we increase the number of weight vectors, the classification accuracy tends, on average, to decrease. This behavior shows that our framework might benefit from the limited number of weight vectors since the slowest components express the essential characteristics varying over time. Given that our method achieved the best performance with $p = 10$, we used this value for the following experiments.

5.3.2. PCA contribution coefficient

To better comprehend the effects of the PCA contribution coefficient ($\eta = \frac{1}{T} \sum_i \|\dot{x}_i\|_2^2$) in our SFS performance along with the mutual subspace methods, we perform classification on the Cambridge gesture and the KTH action datasets while varying the value of η , as these are the standard datasets for action recognition.

Experimental setup: We fixed the number of weight vectors to 10 and varied η to observe which value leads to the best performance across these datasets. For the Cambridge gesture dataset, we use the ranges for the mutual subspace methods hyperparameters from the

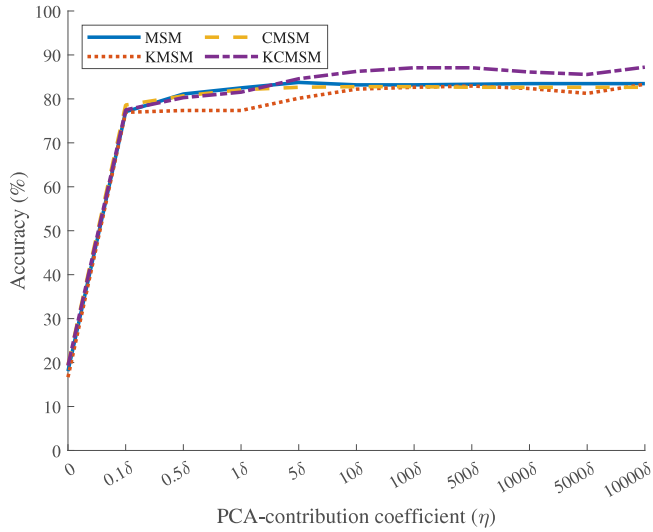


Fig. 8. Performance evaluation of our methods on the Cambridge gesture dataset with the variation of the PCA-contribution coefficient (η), where $\delta \triangleq \frac{1}{T} \sum_i \|\mathbf{x}_i\|_2^2$. This graph shows that lower values of $\eta \leq 10\delta$ generate unstable performances, while values of $\eta \geq 10\delta$ tend to stabilize the classification result.

Table 3

Comparison of classification accuracies (%) for $\eta = 10\delta$ and $\eta = 100\delta$ of our framework on the KTH action dataset, where $\delta \triangleq \frac{1}{T} \sum_i \|\mathbf{x}_i\|_2^2$. For this experiment, we used 10 weight vectors ($p = 10$). These results suggest that, on average, the methods had a better performance with $\eta = 10\delta$. It also indicates that our subspace is robust to hyper-parameter changes as the performance was only slightly different after changing η .

PCA contribution coefficient (η)	MSM	KMSM	CMSM	KCMSM
10δ	72.56	72.18	72.23	73.84
100δ	73.09	72.18	71.84	73.68

previous section. For the KTH action dataset, we performed 10-fold cross-validation with 10 subjects for training and 15 for testing for each of the six classes. Additionally, the dimension of the class subspace S_c varied from 20 to 35 with intervals of 5, the dimension of the input subspace S_{in} varied from 2 to 10, with intervals of 2. For the kernel methods, we varied the Gaussian kernel parameter σ^2 from 0.4 to 0.9 in increments of 0.1. For CMSM, the GDS dimension $\mathcal{G}$ ranged from 2 to 10 in intervals of 2, while the GDS dimension in KCMSM varied from 100 to 200 in increments of 10. We report the results for the combination of hyperparameters that achieved the best average performance across all 10 folds, according to the grid search.

Results and discussion: Fig. 8 shows classification accuracy as we vary η for the Cambridge dataset. In the graph, we use $\delta \triangleq \frac{1}{T} \sum_i \|\mathbf{x}_i\|_2^2$ to simplify the notation of the coefficient η variation. The results suggest that the performance improves as we increase the coefficient in the interval $\eta < 10\delta$. However, when $\eta \geq 10\delta$, the methods tend to stabilize their performance. To determine the most suitable value of η for our task, we further experiment on the KTH action dataset using $\eta = \{10\delta, 100\delta\}$ as both achieved comparable performance, as shown in Fig. 8. The results in Table 3 show that the performance of our methods with $\eta = 10\delta$ is, on average, 72.70%, which is slightly better than 72.69%, achieved with $\eta = 100\delta$. In general, our results suggest, independently of the dataset, that changing η inside this range might not significantly affect our performance. Based on our results, our next experiments will employ only $\eta = 10\delta$, alongside $p = 10$.

5.4. Effectiveness of SFS representation on subspace-based methods

In this experiment, we investigate the efficacy of using the regularized SFA representation in our slow feature subspace. To understand

this effect, we compare the performance of various subspace-based methods using subspaces modeled from raw data (i.e., no time representation), the Hankel matrix, RTW, the original slow features of SFA, the PCA-SFA video descriptor, and our proposed video descriptor.

Experimental setting: In this experiment, we used two different datasets: the KTH action dataset (KTH) and the Chinese sign language dataset (CSL). On KTH, we performed 10-fold cross-validation by setting 10 subjects for training and 15 for testing for each of the six classes. On CSL, each video in grayscale was resized to 38×24 pixels, leading to a 912-dimensional vector for each frame. Additionally, we performed 10-fold cross-validation on CSL, splitting it into 300 training videos, 100 validation videos, and 100 test videos. For both datasets, we report the mean accuracy across the best results in each fold in accordance with the grid search.

We considered the following time representations due to their relevance in the subspace representation field:

- **None:** We model the subspaces using PCA, from vectors with grayscale values of pixels in each frame, thus not including any explicit temporal relation between them.
- **Hankel:** The subspaces are modeled from the trajectory matrix, according to Gatto et al. (2017). We only considered a sliding window of size two.
- **RTW:** TE features proposed in RTW (Suryanto et al., 2016). We only considered a concatenation factor of 2 and sampled 10 TE features per video.
- **SFA:** We model the subspaces from the slow features proposed in the original SFA (Wiskott & Sejnowski, 2002). We only considered the 10 slowest features.
- **PCA-SFA:** The video descriptor proposed by Kobayashi (2017).
- **SFS:** Our proposed slow feature subspace, described in Section 4. We only considered the weight vectors corresponding to the 10 slowest features.

We evaluate the action recognition performance of mutual subspace methods discussed in Section 3.3. For KTH, we used the same hyperparameters used in Section 5.3.2 to determine the subspaces' dimensions. For CSL, the class subspace S_c dimension varied from 20 to 35 with intervals of 5, and the input subspace S_{in} dimension changed from 2 to 10, with intervals of 2. For the kernel methods, the Gaussian kernel parameter σ^2 varied from 0.4 to 0.9 in increments of 0.1. For CMSM, the GDS $\mathcal{G}$ dimension ranged from 20 to 35 in intervals of 5, while it varied from 200 to 350 in increments of 10 in KCMSM.

Specifically for the PCA-SFA features, as this video descriptor results in a single vector per video, instead of the mutual subspace methods, we evaluated using only the subspace method (SM). We also tested using only the first slowest feature of SFA.

Results and discussions: The results of the comparisons are summarized in Table 4. First, we can see that our proposed SFS could significantly improve the performance compared to not using time information (None) on both datasets. Our results outperformed the Hankel matrix and RTW representations, which may be due to their limitations in constructing an effective representation with a smaller number of concatenated vectors. Furthermore, we observe that previous representations based on the slow feature analysis (SFA and PCA-SFA) performed poorly, while our SFS achieved the best performance with all MSMs, demonstrating that exploring the structure of the slow feature's distribution through SFS is more efficient. Concretely, compared to no time representation, SFS improved accuracy by an average of 2.5% on the KTH dataset and 15.9% on the CSL dataset.

Moreover, we can see that KCMSM achieved the best results for the KTH dataset, while MSM performed the best on the Chinese sign language dataset. However, the results with different datasets were significantly different. This discrepancy could be explained by KTH being much simpler than the CSL.

To better visualize the performance of our SFS, we looked at the confusion matrices obtained for different features with KCMSM

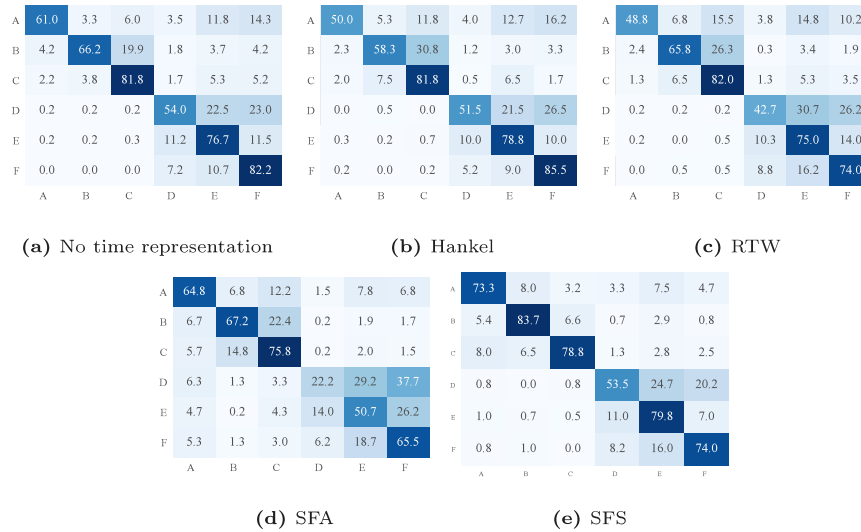


Fig. 9. Confusion matrices of KCMSM on the KTH dataset. Number represent the percentage of predictions attributed to each class, rows represent the predictions, while the columns represent the ground truth.

Table 4

Overview of our results on the KTH action and Chinese sign language (CSL) datasets. We report the mean accuracy (%) across the best results in each fold. The best result for each dataset is indicated in bold. Compared to using no time information (Raw), SFS improved the performance of the MSMs in about 2.5% on the KTH and in 15.9% on the CSL dataset.

Dataset	Time representation	SM	MSM	KMSM	CMSM	KCMSM
KTH	None	–	69.86	70.20	70.51	70.31
	Hankel	–	70.42	69.89	70.34	70.09
	RTW	–	64.61	64.52	68.25	67.50
	SFA	41.74	65.44	54.28	66.44	59.96
	PCA-SFA	47.55	–	–	–	–
	SFS (ours)	–	72.56	72.18	72.23	73.84
CSL	None	–	30.40	30.80	25.50	27.80
	Hankel	–	30.90	30.60	26.70	28.20
	RTW	–	33.30	32.20	33.10	32.80
	SFA	21.10	37.60	20.70	36.90	23.60
	PCA-SFA	25.10	–	–	–	–
	SFS (ours)	–	48.40	44.60	45.70	39.40

on KTH. Fig. 9 shows these matrices, with no time representation (Fig. 9(a)), Hankel matrix (Fig. 9(b)), RTW (Fig. 9(c)), SFA (slow features) (Fig. 9(d)), and SFS (Fig. 9(e)). These matrices show the percentage of predictions attributed to each class: (A) boxing, (B) hand-clapping, (C) handwaving, (D) running, (E) jogging and (F) walking. Rows represent the predictions, while the columns represent the ground truth.

We can see that SFS performed better than slow features for all classes. When compared to no time representation, although SFS performed slightly worse in some classes, such as in (C) and (F), our method significantly improved the accuracy in (A), (B), and (E), achieving the best performance among all. In comparison to the Hankel matrix and RTW representations, our SFS outperformed those methods in classes (A), (B), (D), and (E), while RTW achieved the highest accuracy only in class (C) and Hankel, in class (F).

Additionally, our approach demonstrated a reduction in misclassifications between certain classes. Specifically, it addresses the misclassification of class (A) as class (C) and (F), as well as class (B) being misclassified as class (C). However, the recognition of classes (D) running, (E) jogging, and (F) walking poses a challenge for all approaches. These classes exhibit similar actions at different speeds,

with running being a faster version of jogging, and jogging being a faster version of walking. As a result, we hypothesize that the slowly varying characteristics over time tend to be comparable among these classes in our approach. However, this limitation is not unique to our representation, as similar behavior was observed in other investigated representations.

Moreover, we also analyzed the confusion matrices of MSM obtained by SFA (slow features) and SFS representation for the CSL dataset. Fig. 10 shows the confusion matrix for the slow features (Fig. 10(a)) and for our SFS representation (Fig. 10(b)). These matrices show percentages of per-class predictions to classes: (1) goal, (2) clue, (3) public welfare, (4) development, (5) composition, (6) short, (7) swelling, (8) younger brother, (9) grandmother and (10) soda. In summary, although the SFS representation could not increase the performance of classes (4) and (10), it increased the predictions in all the remaining classes, where its biggest improvements were in classes (3), (8), and (9).

Although SFS is derived from the regularized version of SFA, methods utilizing SFS for representation have shown superior learning capabilities compared to SFA, even with limited data. In this experiment, utilizing only raw data, the classification accuracy (Table 4) on the CSL dataset and the class-level results are comparatively lower than the results obtained for the KTH dataset. This performance difference can be attributed to the complexity of the sign language dataset. Sign language recognition poses significant challenges in action recognition since a single sign can involve multiple hand shapes, movements, and facial expressions, which can vary based on an individual's interpretation of the sign. Therefore, sign language recognition can be highly challenging to methods not specifically tailored for this task.

Overall, these confusion matrices showed that our SFS could improve the results of the subspace-based methods at a class level without resorting to vector concatenations as in RTW, Hankel, and PCA-SFA representations and without additional operations to extract the slow features by SFA. These results demonstrates that our subspace builds a compact and powerful temporal representation.

5.5. Comparison against shallow handcrafted time representations

Once we have established the efficacy of SFS against other time representations for subspaces, we are curious to see where SFS stands against other popular shallow handcrafted time representations for videos.

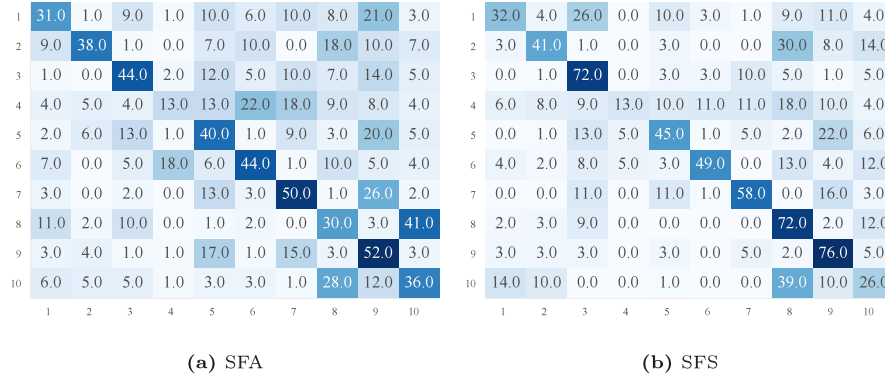


Fig. 10. Confusion matrices of MSM for representations that obtained the highest prediction results on the CSL dataset. Number represent the percentage of predictions attributed to each class, rows represent the predictions, while the columns represent the ground truth.

In general, it is common to use a combination of local feature detectors and descriptors to represent video temporal and spatial information. In this experiment, we follow Wang et al. (2009) and compare to various combinations of the Harris3D (Willems et al., 2008), Hessian (Willems et al., 2008), and Dense (Wang et al., 2009) local feature detectors, with the HOG/HOF (Laptev et al., 2008), and HOG3D (Klaeser et al., 2008) descriptors. We also compare our performance against other SFA-based representations, such as the one-layer SFA (Zhang & Tao, 2012) and DL-SFA (Sun et al., 2014).

All of the methods mentioned follow a similar workflow, wherein a data representation is first constructed, followed by the application of a classifier such as SVM, GDA (Hamm & Lee, 2008), eGDA (Souza et al., 2020), or MSMs to obtain the final result.

Experimental setting: We used the UCF Sports dataset (UCF), which has been widely utilized to evaluate the performance of the above-mentioned features. We follow the same experimental setting as the other methods, performing a leave-one-out cross-validation (LOOCV) scheme and report the mean average across all folds with hyperparameters defined following the grid search.

Our framework used the following parameters: the class subspace S_c dimension were set to 20, 30 and 40, while the test subspace S_{in} dimension varied from 2 to 10, with step 1. For CMSM and KCMSM, the GDS dimension varied from 50 to 60 in intervals of 5. Regarding KMSM and KCMSM, the Gaussian-Kernel bandwidth parameter varied from 0.4 to 0.9 in increments of 0.1. Finally, results for the local feature detectors and descriptors were taken from the original study paper (Wang et al., 2009), while the results for one-layer SFA and DL-SFA were taken from Sun et al. (2014).

Results and discussions: Results are shown in Table 5. We can see that despite the handcraft features performing well, performance using SFA, in general, was better. This result shows clearly how SFA can help in the task of action recognition. When using RTW, another time representation method for subspaces, we can see it performs relatively well along with eGDA, despite not performing better than SFA with KCMSM.

Nevertheless, it is clear that exploring the distribution of slow features by combining SFA with the MSMs leads to better results over concatenation with SVM. Ultimately, exploring this distribution through a set of weight vectors, as in SFS, led to the best results.

5.6. Comparison against deep learning time representations

In this section, we investigate how the SFS performs against more recent time representation based on deep learning. We are particularly curious to see how SFS performs when using features extracted from pre-trained convolutional neural networks, i.e., CNN features, as the subspace-based methods have been shown to work well with such features to solve action recognition related tasks, such as gait recognition

Table 5

Comparison results on the UCF Sports dataset between the SFS along with MSMs and shallow handcrafted time representations. We report the average classification accuracy across all folds. Results in bold show the best result for each category of representations. Results for handcrafted features were taken from Wang et al. (2009), while the results for one-layer SFA and DL-SFA were taken from Sun et al. (2014).

Representation	Classification method	Accuracy (%)
Hessian + ESURF	SVM	77.30
Harris3D + HOG/HOF		78.10
Hessian + HOG/HOF		79.30
Dense + HOF		82.60
Cuboids + HOG3D		82.90
Dense + HOG3D		85.60
One-layer SFA	SVM	79.80
DL-SFA		86.60
SFA	MSM	78.67
	KMSM	70.00
	CMSM	87.33
	KCMSM	97.33
RTW	GDA	67.33
	eGDA	92.67
SFS (ours)	MSM	72.67
	KMSM	71.33
	CMSM	92.00
	KCMSM	98.00

(Sakai et al., 2019). Furthermore, we also seek to understand if the SFS would work well with features extracted from more complex and modern architectures, such as 3D-convolutional networks and Transformers. Therefore, in this experiment, we compare the performance of the SFS along with the MSMs when using raw features, as well as with CNN features and other deep features.

5.6.1. Slow feature subspaces from CNN features

We first start by analyzing the performance of the slow feature subspace when modeled with CNN features. We refer to “CNN features” the features extracted by forwarding an image through a CNN and extracting its representation from one of the hidden layers. Specifically, we use VGG-19 (Simonyan & Zisserman, 2015), a widely used convolutional neural network for image classification, pre-trained on the ImageNet (Deng et al., 2009) and assess the SFS performance when using these CNN features, compared to raw features. Finally, we also compare our SFS along with MSMs to a fine-tuned VGG-19.

Experimental setting: We performed this experiment using the Chinese sign language dataset. This dataset was selected as it posed a challenge to the methods analyzed in Section 5.4 when we considered only raw data. For evaluation, we followed the same cross-validation scheme used for this dataset in previous sections and report the average classification accuracy across all folds.

Table 6

Results on the Chinese sign language dataset with the comparison between the SFS and the MSMs when using raw data and CNN features extracted from VGG-19. We report the mean accuracy (%) across all folds. Results in bold indicate the best performance.

CNN training	Time representation	Method	Data	
			Raw	CNN
Pre-trained	None	MSM	30.40	60.80
		KMSM	30.80	58.90
		CMSM	25.50	58.60
		KCMSM	27.80	61.40
		SM	21.10	29.10
	SFA	MSM	37.60	57.60
		KMSM	20.70	55.50
		CMSM	36.90	51.90
		KCMSM	23.60	54.90
	PCA-SFA	SM	25.10	46.50
		MSM	48.40	72.00
	SFS (ours)	KMSM	44.60	74.80
		CMSM	45.70	62.30
		KCMSM	39.40	73.10
Fine-tuned	None	VGG-19	30.70	

As for features, we used two different types:

- **Raw:** Vectors containing the grayscale pixel values of each frame. Each video was resized to 38×24 pixels, leading to a 912-dimensional raw feature vector for each frame.
- **CNN:** For each video, we processed each frame through a pre-trained VGG-19 (Simonyan & Zisserman, 2015), extracting the hidden representations from the fifth block, on top of which we applied global average pooling to reduce the feature dimension, obtaining a 512-dimensional feature vector for each video frame.

For each mutual subspace method, the class subspace S_c dimension varied from 20 to 35 with intervals of 5, and the input subspace S_{in} dimension ranged from 2 to 10, 2-by-2. For the kernel methods, the Gaussian kernel parameter σ^2 varied from 0.4 to 0.9 in increments of 0.1. For CMSM, the GDS $\mathcal{G}$ dimension ranged from 20 to 35 in intervals of 5, while it varied from 200 to 350 in increments of 10 in KCMSM. We used the grid search to determine the hyperparameters that achieve the best results in each fold.

For VGG-19's fine-tuning, we defined the learning rate (LR) as $1e-5$ after performing a small experiment in one fold using the LR finder (Smith, 2017). We also utilized stochastic gradient descent (SGD) as an optimizer, and trained for 50 epochs with a batch size of 32. During fine-tuning, we only trained the block immediately before the FC layers. Since VGG-19 is an image-based network, we assigned the labels from each frame as the same label of its video. For classification, we performed a majority vote scheme, where the video was assigned to the class to which the majority of its frames were classified into.

Results and discussions: Table 6 shows our results. In general, we can observe that all methods, regardless of the time representation, benefit from using CNN features. However, we can see that using no time representation along with the MSMs led to overall better performance than using the original SFA and the PCA-SFA descriptors, even when CNN features are used. Moreover, we can observe that, despite the fine-tuning, VGG-19 by itself did not perform very well.

However, it is interesting to see that using the SFS time representation along with pre-trained CNN features achieved the best performance among all methods, with a drastic improvement when compared with the raw data. This result shows that the SFS can efficiently explore the temporal relation from CNN features of a video's frames.

5.6.2. Slow feature subspaces from deep features

We further investigate how the SFS performs when using features extracted by more complex and modern architectures, such as 3D

Table 7

Hyperparameters utilized in the fine-tuning of the VGG19, I3D, TimeSformer and Swin-B models.

Parameter	VGG19	I3D	TimeSformer	Swin-B
Backbone	VGG19	3D Resnet 50	TimeSformer	SwinTransformer3D
Batch size	32	10	16	8
Learning rate	$1e-5$	0.005	0.01	0.01
Optimizer	SGD	SGD	SGD	AdamW
Epochs	50	20	8	8
Momentum	0.9	0.9	0.9	–
Weight decay	None	$1e-4$	$1e-4$	0.5

Table 8

Comparison of classification accuracies (%) among recent neural network architectures and our proposed framework on the UCF sports dataset. Results in bold indicate the best performances for pre-trained and fine-tuned networks.

Network Training	Time representation	Method	Data		
			Raw	I3D	TimeSformer
Pre-trained	SFS (ours)	MSM	64.67	90.67	89.33
		KMSM	74.00	89.33	88.00
		CMSM	67.33	90.00	92.00
		KCMSM	73.33	90.00	90.00
Fine-tuned	3D-conv Transformer	I3D		83.33	
		TimeSformer		89.33	

convolutional neural networks and Transformers. Specifically, we use pre-trained versions of I3D (Carreira & Zisserman, 2017; Wang et al., 2018) and the TimeSformer (Bertasius et al., 2021). The I3D model inflates the filters and pooling kernels of 2D pre-trained models to a 3D architecture, leading to a natural spatio-temporal classifier. Moreover, TimeSformer leverages the self-attention mechanism and the temporal modeling by using positional encoding of frames analogous to the learning strategy used in natural language processing transformers.

Experimental setting: For the following experiments with UCF Sports, we performed stratified 10-fold cross-validation, where we split 135 samples for training and 15 for evaluation.

Following the previous experiments, we also extracted hidden representations from pre-trained versions of these neural networks. Both networks were pre-trained on the Kinetics 400 dataset (Kay et al., 2017). In our experiments, we considered the I3D with 3D Resnet 50 backbone, and for TimeSformer, we utilized the joint space and time attention architecture.¹ Further information about the hyperparameters used for fine-tuning is reported in Table 7.

These networks receive a sequence of frames as input and generate one single feature vector before classification, as they were designed for videos. Since the mutual subspace methods require a set of features for each video, instead of processing the whole videos at once, we employ a simple data augmentation technique to obtain such set: we use the “SampleFrames” function of the MMAction2 (Contributors, 2020) toolbox, where we defined 10 clips to be sampled from the original video. After that, the number of crops per video was set to 3 using the “ThreeCrop” function, resulting in 30 samples for each video. This process ultimately results in 30 features representing each video.

As for the hyperparameters of our framework, the dimension of the class subspace S_c was fixed in 50, while the dimension of the input subspace S_{in} was 6. Besides, in the CMSM, the GDS dimension $\mathcal{G}$ was set to 10, while it was set to 400 in KCMSM. For the kernel methods, the Gaussian kernel parameter σ^2 was set to 100.

Results and discussions: Table 8 shows our results. We report the mean accuracy across all folds. Here, we can also observe that

¹ We used the pre-trained weights “i3d_r50_32x2x1_100e_kinetics400_rgb” for I3D, and “timesformer_jointST_8x32x1_15e_kinetics400_rgb” for TimeSformer, both are available in MMAction2 toolbox.

Table 9

Comparison of classification accuracies (%) among state-of-the-art methods and our framework on the UCF101 dataset. In our framework, we utilized the pre-trained features of the Video Swin Transformer (Swin-B). Our results indicate that our method performs competitively with the state-of-the-art methods, with only an approximate difference of 4 percentage points in performance between them.

Network Training	Time representation	Method	Accuracy (%)
Swin-B pre-trained features	SFS (ours)	MSM	95.47
		KMSM	94.66
		CMSM	95.58
		KCMSM	95.62
Fine-tuned	Transformer	Swin-B	88.55
	Two 3D streams	Two-Stream I3D	97.80
	Joint learning	OmniSource	98.69
	VLM	BIKE	98.90
	Transformer	VideoMAE V2	99.60

SFS significantly improves performance when using the pre-trained features, where the best results were achieved using TimeSformer features. Furthermore, we can also observe that SFS with pre-trained features performed better than the fine-tuned versions of I3D and TimeSformer on the analyzed dataset.

To understand if these differences are statistically significant, we performed several paired two-sample t-tests between the results of each neural network and our framework using their extracted features.

On our first set of paired two-sample t-tests, we compared the results of the fine-tuned I3D and SFS using I3D pre-trained features. The t-tests revealed that our methods achieved significant performance differences with p-values of 0.01744 (MSM), 0.0229 (CMSM), 0.0239 (KMSM), and 0.01497 (KCMSM). Our second paired two-sample t-test, where we compared the results of the fine-tuned TimeSformer and our methods with its pre-trained features indicated that we cannot affirm with more than 95% confidence that the methods achieved significant differences from each other with p-values of 0.9996 (MSM), 0.1039 (CMSM), 0.5551 (KMSM) and 0.5911 (KCMSM).

These results demonstrate that the SFS can work well with such pre-trained deep features, achieving performance comparable to fine-tuning, as with TimeSformer, or even outperforming fine-tuning, as with I3D. These results along with the ones using CNN features suggest that SFS along with the mutual subspace methods can offer a good alternative to fine-tuning.

5.7. Comparisons with state-of-the-art methods

In this section, we compare our performance to recent state-of-the-art methods that were fine-tuned on the UCF101 dataset. The purpose of this experiment is to determine if our method can achieve competitive results without the need for fine-tuning the neural network, thereby saving both time and computational resources. For this comparison, we extracted deep features from Video Swin Transformer (Liu et al., 2022) that was pre-trained on the Kinetics 400 dataset. We utilized the official source code of Video Swin Transformer and its pre-trained weights, which are publicly available on GitHub.²

Experimental setting: For the following experiments, we report the accuracy across the 3 train/test splits of UCF101, which is a common experimental setting for this dataset.

Results for VideoMAE V2 (Wang et al., 2023), BIKE (Wu et al., 2023), OmniSource (Duan et al., 2020) and Two-Stream I3D (Carreira & Zisserman, 2017) were taken from their respective research papers. These methods were initially pre-trained on the Kinetics 400 dataset, except for VideoMAE V2, which underwent pre-training using multiple

datasets. Subsequently, fine-tuning was performed on these models on the UCF101 dataset.

These methods were selected due to their relevance and results in action recognition. Besides, each one of them incorporates state-of-the-art techniques for learning such as transformers (Liu et al., 2022; Wang et al., 2023), vision-language models (VLM) (Wu et al., 2023), two 3D streams (Carreira & Zisserman, 2017) and even employs multiple neural networks through joint learning (Duan et al., 2020). By leveraging these mechanisms, these models have demonstrated notable learning ability, resulting in high classification performance.

Following the previous experiments, we also extracted pre-trained features from the last layer of the Video Swin Transformer frozen backbone. Specifically, we use the base architecture (Swin-B) of this neural network, which was pre-trained in the Kinetics 400. As in the previous experiment, we extracted 30 feature vectors for each video using the same data augmentation technique, where each feature vector has 1024 dimensions. The hyperparameters used in the fine-tuning are summarized in Table 7.

Regarding the hyperparameters of our framework, the dimension of the class subspace S_c was fixed in 10, while the dimension of the input subspace S_{in} was 10 as well. Besides, the GDS dimension G was set to 5 in the CMSM, while it was set to 1000 in KCMSM. For the kernel methods, the Gaussian kernel parameter σ^2 was set to 100.

Results and discussions: We report our mean accuracy in Table 9 and compare our results to several state-of-the-art methods.

Our findings indicate that by utilizing the pre-trained features extracted from the frozen backbone of the Swin-B model, we achieved a classification accuracy of approximately 7 percentage points higher than the accuracy obtained through fine-tuning the Swin-B model.

Remarkably, our results demonstrate that our methods achieve competitive performance compared to the state-of-the-art methods on the UCF101 dataset. Although other methods achieved higher performance, they also have several limitations. The learning regime of VideoMAE V2 heavily relies in more than a million videos for training its billion-level parameters, making it less effective in scenarios with limited data. Additionally, the BIKE method requires textual data, which may not be available in all video datasets. The Omnisource method enables shared video representations by utilizing a joint learning framework, which requires increased computational resources. The Two-Stream I3D model relies on RGB and optical flow data, which introduces additional computational complexity. These limitations highlight the contextual constraints of these models, where their requirements must be fulfilled for optimal performance. Otherwise, their effectiveness may be compromised.

In our experiment, we specifically leverage the pre-trained features extracted from the network and utilize them for training our method in the action recognition task. In this way, KCMSM with SFS achieved a performance that is only 4.4 percentage points lower than the best performing method, VideoMAE V2. This strategy allows us to achieve competitive performance with state-of-the-art methods without the computational burden and data requirements associated with fine-tuning a neural network.

6. Conclusion and future work

This paper proposed a new video representation for action recognition called slow feature subspace (SFS). SFS leverages slow feature analysis (SFA) to extract slowly varying features from videos, addressing the issue of lost temporal relationships present in conventional subspace-based methods. Our representation is a subspace generated from the slowest weight vectors of SFA, which was incorporated into mutual subspace methods, significantly improving performance. Moreover, SFS performance with deep learning features outperformed fine-tuned neural network models by using only pre-trained features and achieved results competitive to state-of-the-art methods, suggesting that SFS can even be a potential alternative to fine-tuning, which is not always

² <https://github.com/SwinTransformer/Video-Swin-Transformer>.

feasible in practice. The main limitation of SFS is the inclusion of two new hyperparameters to be tuned; however, as shown, SFS exhibits robustness to these hyperparameters changes. Finally, our future work includes developing an end-to-end approach, where our SFS is built while a deep neural network is being trained.

CRedit authorship contribution statement

Suzana Rita Alves Beleza: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Erica K. Shimomoto:** Methodology, Formal analysis, Visualization, Writing – review & editing. **Lincon S. Souza:** Methodology, Formal analysis, Visualization, Writing – review & editing. **Kazuhiro Fukui:** Conceptualization, Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The datasets utilized are publicly available.

Acknowledgments

This work was supported by JSPS KAKENHI, Japan Grant No. JP22K17960; JSPS KAKENHI, Japan Grant No. 19H04129; JSPS KAKENHI, Japan Grant No. 21K18481 and the Japanese Ministry of Education, Culture, Sports, Science and Technology (MEXT) scholarship, Japan.

References

- Afriat, S. N. (1957). Orthogonal and oblique projectors and the characteristics of pairs of vector spaces. In *Mathematical proceedings of the cambridge philosophical society*, Vol. 53 (pp. 800–816). <http://dx.doi.org/10.1017/S0305004100032916>.
- Arnab, A., Dehghani, M., Heigold, G., Sun, C., Lučić, M., & Schmid, C. (2021). ViViT: A video vision transformer. In *2021 IEEE/CVF international conference on computer vision* (pp. 6816–6826). <http://dx.doi.org/10.1109/ICCV48922.2021.00676>.
- Aslan, M. F., Durdu, A., & Sabanci, K. (2020). Human action recognition with bag of visual words using different machine learning methods and hyperparameter optimization. *Neural Computing and Applications*, 32(12), 8585–8597. <http://dx.doi.org/10.1007/s00521-019-04365-9>.
- Batalo, B., Souza, L. S., Gatto, B. B., Sogi, N., & Fukui, K. (2022). Temporal-stochastic tensor features for action recognition. *Machine Learning with Applications*, 10, Article 100407. <http://dx.doi.org/10.1016/j.mlwa.2022.100407>.
- Beleza, S. R., & Fukui, K. (2021). Slow feature subspace for action recognition. In *Pattern recognition. ICPR international workshops and challenges: virtual event, January 10–15, 2021, proceedings, part III* (pp. 702–716). http://dx.doi.org/10.1007/978-3-030-68796-0_51.
- Berkes, P., & Wiskott, L. (2005). Slow feature analysis yields a rich repertoire of complex cell properties. *Journal of Vision*, 5(f6), 579–602. <http://dx.doi.org/10.1167/5.6.9>.
- Bertasius, G., Wang, H., & Torresani, L. (2021). Is space-time attention all you need for video understanding? In *ICML*, Vol. 2 (p. 4).
- Carreira, J., & Zisserman, A. (2017). Quo vadis, action recognition? a new model and the kinetics dataset. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 6299–6308). <http://dx.doi.org/10.1109/CVPR.2017.502>.
- Cheok, M. J., Omar, Z., & Jaward, M. H. (2019). A review of hand gesture and sign language recognition techniques. *International Journal of Machine Learning and Cybernetics*, 10(1), 131–153. <http://dx.doi.org/10.1007/s13042-017-0705-5>.
- Contributors, M. (2020). OpenMMLab's next generation video understanding toolbox and benchmark. <https://github.com/open-mmlab/mmdetection>.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition* (pp. 248–255). <http://dx.doi.org/10.1109/CVPR.2009.5206848>.
- Donahue, J., Anne Hendricks, L., Guadarrama, S., Rohrbach, M., Venugopalan, S., Saenko, K., & Darrell, T. (2015). Long-term recurrent convolutional networks for visual recognition and description. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2625–2634).
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2020). An image is worth 16 × 16 words: Transformers for image recognition at scale. CoRR [abs/2010.11929](https://arxiv.org/abs/2010.11929). arXiv:2010.11929. URL: <https://arxiv.org/abs/2010.11929>.
- Duan, H., Zhao, Y., Xiong, Y., Liu, W., & Lin, D. (2020). Omni-sourced webly-supervised learning for video recognition. In *Computer vision – ECCV 2020* (pp. 670–688). http://dx.doi.org/10.1007/978-3-030-58555-6_40.
- Fan, K., Wang, P., & Zhuang, S. (2019). Human fall detection using slow feature analysis. *Multimedia Tools and Applications*, 78(7), 9101–9128. <http://dx.doi.org/10.1007/s11042-018-5638-9>.
- Franzius, M., Sprekeler, H., & Wiskott, L. (2007). Slowness and sparseness lead to place, head-direction, and spatial-view cells. *PLoS Computational Biology*, 3(8), Article e166. <http://dx.doi.org/10.1371/journal.pcbi.0030166>.
- Fukui, K. (2020). Subspace methods. In *Computer vision: a reference guide* (pp. 1–5). http://dx.doi.org/10.1007/978-3-030-03243-2_708-1.
- Fukui, K., & Maki, A. (2015). Difference subspace and its generalization for subspace-based methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 37(11), 2164–2177. <http://dx.doi.org/10.1109/TPAMI.2015.2408358>.
- Fukui, K., Sogi, N., Kobayashi, T., Xue, J.-H., & Maki, A. (2023). Discriminant feature extraction by generalized difference subspace. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(2), 1618–1635. <http://dx.doi.org/10.1109/TPAMI.2022.3168557>.
- Fukui, K., Stenger, B., & Yamaguchi, O. (2006). A framework for 3D object recognition using the kernel constrained mutual subspace method. In *Asian conference on computer vision* (pp. 315–324). http://dx.doi.org/10.1007/11612704_32.
- Fukui, K., & Yamaguchi, O. (2005). Face recognition using multi-viewpoint patterns for robot vision. In *Robotics research. The eleventh international symposium* (pp. 192–201). Springer.
- Gatto, B. B., Bogdanova, A., Souza, L. S., & dos Santos, E. M. (2017). Hankel subspace method for efficient gesture representation. In *2017 IEEE 27th international workshop on machine learning for signal processing* (pp. 1–6). <http://dx.doi.org/10.1109/MLSP.2017.8168114>.
- Hamm, J., & Lee, D. D. (2008). Grassmann discriminant analysis: a unifying view on subspace-based learning. In *Proceedings of the 25th international conference on machine learning* (pp. 376–383). <http://dx.doi.org/10.1145/1390156.1390204>.
- Hara, K., Kataoka, H., & Satoh, Y. (2018). Can spatiotemporal 3D CNNs retrace the history of 2D CNNs and ImageNet? In *2018 IEEE/CVF conference on computer vision and pattern recognition* (pp. 6546–6555). <http://dx.doi.org/10.1109/CVPR.2018.00685>.
- Hotelling, H. (1992). Relations between two sets of variates. In *Breakthroughs in statistics* (pp. 162–190). <http://dx.doi.org/10.2307/2333955>.
- Iijima, T., Genchi, H., & Mori, K.-i. (1974). A theory of character recognition by pattern matching method. In *Learning systems and intelligent robots* (pp. 437–450). http://dx.doi.org/10.1007/978-1-4684-2106-4_22.
- Kay, W., Carreira, J., Simonyan, K., Zhang, B., Hillier, C., Vijayanarasimhan, S., Viola, F., Green, T., Back, T., Natsev, P., Suleyman, M., & Zisserman, A. (2017). The kinetics human action video dataset. CoRR [abs/1705.06950](https://arxiv.org/abs/1705.06950). arXiv:1705.06950. URL: <http://arxiv.org/abs/1705.06950>.
- Kim, T.-K., Wong, S.-F., & Cipolla, R. (2007). Tensor canonical correlation analysis for action classification. In *2007 IEEE conference on computer vision and pattern recognition* (pp. 1–8). <http://dx.doi.org/10.1109/CVPR.2007.383137>.
- Klaeser, A., Marszalek, M., & Schmid, C. (2008). A spatio-temporal descriptor based on 3D-gradients. In *Proceedings of the British machine vision conference* (pp. 99.1–99.10). <http://dx.doi.org/10.5244/C.22.99>.
- Kobayashi, T. (2017). Feature sequence representation via slow feature analysis for action classification. In *Proceedings of the British machine vision conference* (pp. 125.1–125.13). <http://dx.doi.org/10.5244/C.31.125>.
- Laptev, I., Marszalek, M., Schmid, C., & Rozenfeld, B. (2008). Learning realistic human actions from movies. In *2008 IEEE conference on computer vision and pattern recognition* (pp. 1–8). <http://dx.doi.org/10.1109/CVPR.2008.4587756>.
- Li, S., Liu, H., Qian, R., Li, Y., See, J., Fei, M., Yu, X., & Lin, W. (2022). TA2N: Two-stage action alignment network for few-shot action recognition. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36–2 (pp. 1404–1411). <http://dx.doi.org/10.1609/aaai.v36i2.20029>.
- Liu, Z., Ning, J., Cao, Y., Wei, Y., Zhang, Z., Lin, S., & Hu, H. (2022). Video swin transformer. In *2022 IEEE/CVF conference on computer vision and pattern recognition* (pp. 3192–3201). <http://dx.doi.org/10.1109/CVPR52688.2022.00320>.
- Liu, T., Zhou, W., & Li, H. (2016). Sign language recognition with long short-term memory. In *2016 IEEE international conference on image processing* (pp. 2871–2875). <http://dx.doi.org/10.1109/ICIP.2016.7532884>.
- Lu, J., Wang, G., & Zhou, J. (2017). Simultaneous feature and dictionary learning for image set based face recognition. *IEEE Transactions on Image Processing*, 26(8), 4042–4054. <http://dx.doi.org/10.1109/TIP.2017.2713940>.
- Maeda, K., & Watanabe, S. (1985). A pattern matching method with local structure. *IEICE Transactions*, 68(3), 345–352.
- Ng, J. Y.-H., Hausknecht, M., Vijayanarasimhan, S., Vinyals, O., Monga, R., & Toderici, G. (2015). Beyond short snippets: Deep networks for video classification. In *2015 IEEE conference on computer vision and pattern recognition* (pp. 4694–4702). <http://dx.doi.org/10.1109/CVPR.2015.7299101>.

- Pu, J., Zhou, W., & Li, H. (2016). Sign language recognition with multi-modal features. In *Pacific rim conference on multimedia* (pp. 252–261). http://dx.doi.org/10.1007/978-3-319-48896-7_25.
- Pu, J., Zhou, W., Zhang, J., & Li, H. (2016). Sign language recognition based on trajectory modeling with hmms. In *International conference on multimedia modeling* (pp. 686–697). http://dx.doi.org/10.1007/978-3-319-27671-7_58.
- Rodriguez, M. D., Ahmed, J., & Shah, M. (2008). Action MACH a spatio-temporal maximum average correlation height filter for action recognition. In *2008 IEEE conference on computer vision and pattern recognition* (pp. 1–8). <http://dx.doi.org/10.1109/CVPR.2008.4587727>.
- Sakai, A., Sogi, N., & Fukui, K. (2019). Gait recognition based on constrained mutual subspace method with cnn features. In *2019 16th international conference on machine vision applications* (pp. 1–6). <http://dx.doi.org/10.23919/MVA.2019.8758035>.
- Sakano, H., & Mukawa, N. (2000). Kernel mutual subspace method for robust facial image recognition. In *KES'2000. Fourth international conference on knowledge-based intelligent engineering systems and allied technologies. Proceedings (Cat. No. 00TH8516), Vol. 1* (pp. 245–248). <http://dx.doi.org/10.1109/KES.2000.885803>.
- Schölkopf, B., Smola, A., & Müller, K.-R. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5), 1299–1319. <http://dx.doi.org/10.1162/089976698300017467>.
- Schuldt, C., Laptev, I., & Caputo, B. (2004). Recognizing human actions: a local SVM approach. In *Proceedings of the 17th international conference on pattern recognition, 2004. ICPR 2004, Vol. 3* (pp. 32–36). <http://dx.doi.org/10.1109/ICPR.2004.1334462>.
- Shan, Y., Zhang, Z., & Huang, K. (2014). Learning skeleton stream patterns with slow feature analysis for action recognition. In *European conference on computer vision* (pp. 111–121). http://dx.doi.org/10.1007/978-3-319-16199-0_8.
- Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *arXiv:1409.1556*. URL: <https://arxiv.org/abs/1409.1556>.
- Smith, L. N. (2017). Cyclical learning rates for training neural networks. In *2017 IEEE winter conference on applications of computer vision* (pp. 464–472). <http://dx.doi.org/10.1109/WACV.2017.58>.
- Sogi, N., Nakayama, T., & Fukui, K. (2018). A method based on convex cone model for image-set classification with cnn features. In *2018 international joint conference on neural networks* (pp. 1–8). <http://dx.doi.org/10.1109/IJCNN.2018.8489151>.
- Soomro, K., & Zamir, A. (2014). Action recognition in realistic sports videos. *Advances in Computer Vision and Pattern Recognition*, 71, 181–208. http://dx.doi.org/10.1007/978-3-319-09396-3_9.
- Soomro, K., Zamir, A. R., & Shah, M. (2012). UCF101: a dataset of 101 human actions classes from videos in the wild. *CoRR abs/1212.0402*. *arXiv:1212.0402*. URL: <http://arxiv.org/abs/1212.0402>.
- Souza, L. S., Gatto, B. B., Xue, J.-H., & Fukui, K. (2020). Enhanced grassmann discriminant analysis with randomized time warping for motion recognition. *Pattern Recognition*, 97, Article 107028. <http://dx.doi.org/10.1016/j.patcog.2019.107028>.
- Souza, L. S., Sogi, N., Gatto, B. B., Kobayashi, T., & Fukui, K. (2023). Grassmannian learning mutual subspace method for image set recognition. *Neurocomputing*, 517, 20–33. <http://dx.doi.org/10.1016/j.neucom.2022.10.040>.
- Sun, L., Jia, K., Chan, T.-H., Fang, Y., Wang, G., & Yan, S. (2014). DL-SFA: Deeply learned slow feature analysis for action recognition. In *2014 IEEE conference on computer vision and pattern recognition* (pp. 2625–2632). <http://dx.doi.org/10.1109/CVPR.2014.336>.
- Suryanto, C. H., Xue, J.-H., & Fukui, K. (2016). Randomized time warping for motion recognition. *Image and Vision Computing*, 54, 1–11. <http://dx.doi.org/10.1016/j.imavis.2016.07.003>.
- Tanaka, S., Okazaki, A., Kato, N., Hino, H., & Fukui, K. (2016). Spotting fingerspelled words from sign language video by temporally regularized canonical component analysis. In *2016 IEEE international conference on identity, security and behavior analysis* (pp. 1–7). <http://dx.doi.org/10.1109/ISBA.2016.7477238>.
- Wang, X., Girshick, R., Gupta, A., & He, K. (2018). Non-local neural networks. In *2018 IEEE/CVF conference on computer vision and pattern recognition* (pp. 7794–7803). <http://dx.doi.org/10.1109/CVPR.2018.00813>.
- Wang, L., Huang, B., Zhao, Z., Tong, Z., He, Y., Wang, Y., Wang, Y., & Qiao, Y. (2023). Videomae V2: Scaling video masked autoencoders with dual masking. *arXiv:2303.16727*. URL: <https://arxiv.org/abs/2303.16727>.
- Wang, H., Ullah, M. M., Klaser, A., Laptev, I., & Schmid, C. (2009). Evaluation of local spatio-temporal features for action recognition. In *BMVC 2009-British machine vision conference* (pp. 124.1–124.11). <http://dx.doi.org/10.5244/C.23.124>.
- Wang, X., Zhang, S., Qing, Z., Tang, M., Zuo, Z., Gao, C., Jin, R., & Sang, N. (2022). Hybrid relation guided set matching for few-shot action recognition. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 19948–19957). <http://dx.doi.org/10.1109/CVPR52688.2022.01932>.
- Watanabe, S. (1967). Evaluation and selection of variables in pattern recognition. *Computer and Information Science II*, 91–122.
- Watanabe, S., & Pakvasa, N. (1973). Subspace method of pattern recognition. In *Proc. 1st. IJCPR* (pp. 25–32).
- Willems, G., Tuytelaars, T., & Van Gool, L. (2008). An efficient dense and scale-invariant spatio-temporal interest point detector. In *Computer vision—ECCV 2008: 10th European conference on computer vision* (pp. 650–663). http://dx.doi.org/10.1007/978-3-540-88688-4_48.
- Wiskott, L. (1999). Learning invariance manifolds. *Neurocomputing*, 26, 925–932. [http://dx.doi.org/10.1016/S0925-2312\(99\)00011-9](http://dx.doi.org/10.1016/S0925-2312(99)00011-9).
- Wiskott, L., & Sejnowski, T. J. (2002). Slow feature analysis: unsupervised learning of invariances. *Neural Computation*, 14(4), 715–770. <http://dx.doi.org/10.1162/089976602317318938>.
- Wu, W., Wang, X., Luo, H., Wang, J., Yang, Y., & Ouyang, W. (2023). Bidirectional cross-modal knowledge exploration for video recognition with pre-trained vision-language models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 6620–6630).
- Yamaguchi, O., Fukui, K., & Maeda, K. (1998). Face recognition using temporal image sequence. In *Proceedings third IEEE international conference on automatic face and gesture recognition* (pp. 318–323). <http://dx.doi.org/10.1109/AFGR.1998.670968>.
- Zhang, Z., & Tao, D. (2012). Slow feature analysis for human action recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34(3), 436–450. <http://dx.doi.org/10.1109/TPAMI.2011.157>.
- Zhang, J., Zhou, W., Xie, C., Pu, J., & Li, H. (2016). Chinese sign language recognition with adaptive HMM. In *2016 IEEE international conference on multimedia and expo* (pp. 1–6). <http://dx.doi.org/10.1109/ICME.2016.7552950>.